

Использование ГПУ в Python

Distributed Memory systems



GPU user:

CUDA
OpenCL
HIP
Open Acc

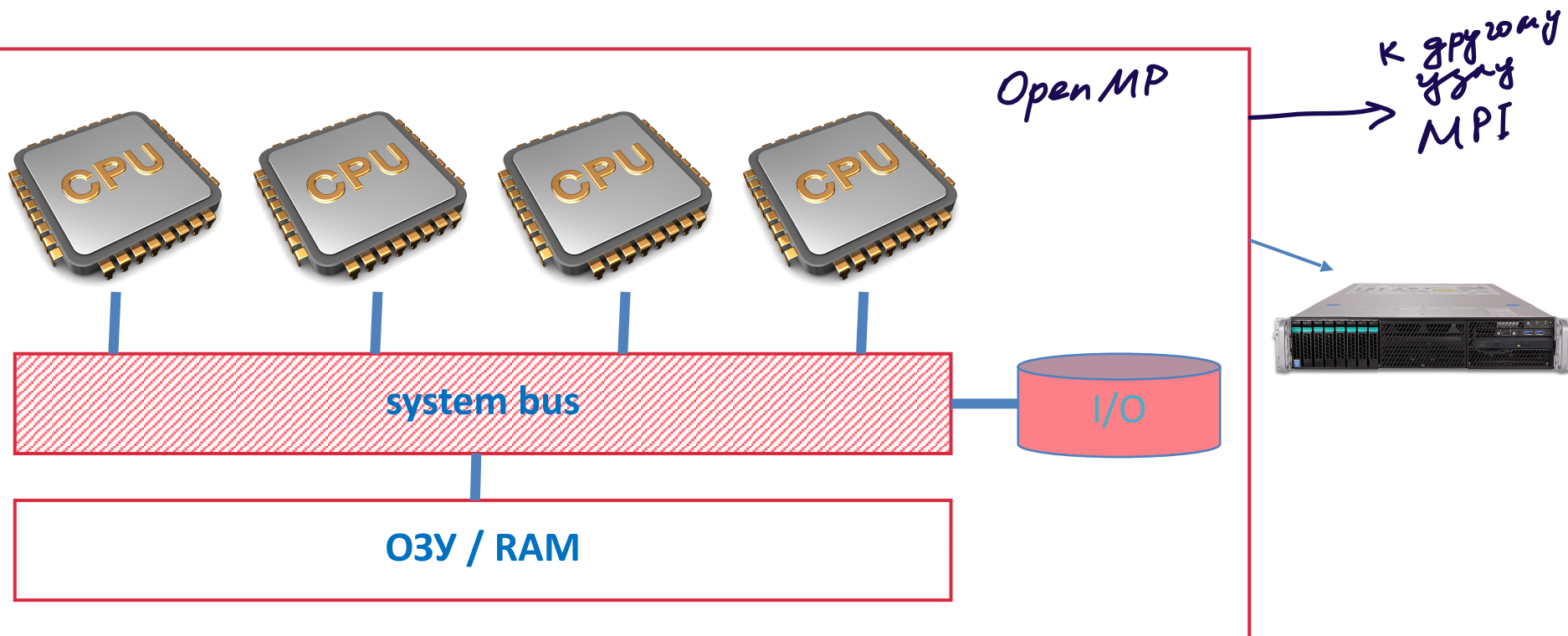
CPU user:

Open MP
Open Acc
pthreads

+ компьютерная сеть

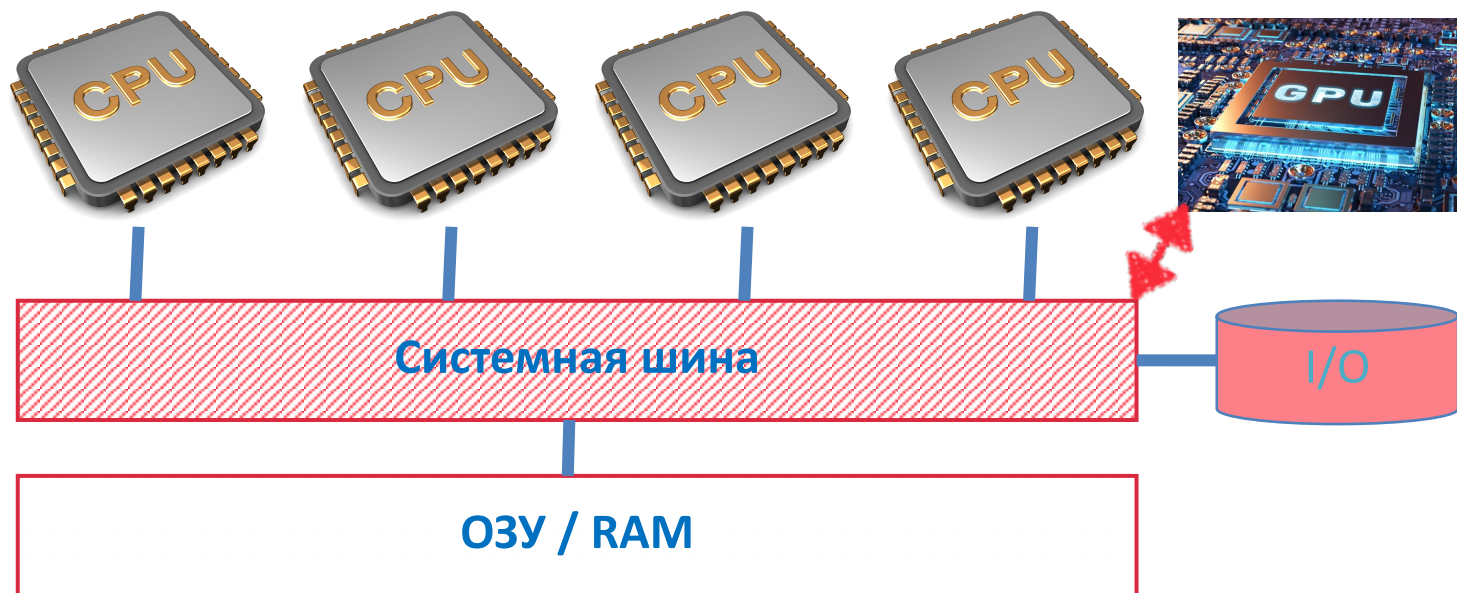
пересылка сообщений между узлами:
Библиотека MPI

shared memory systems

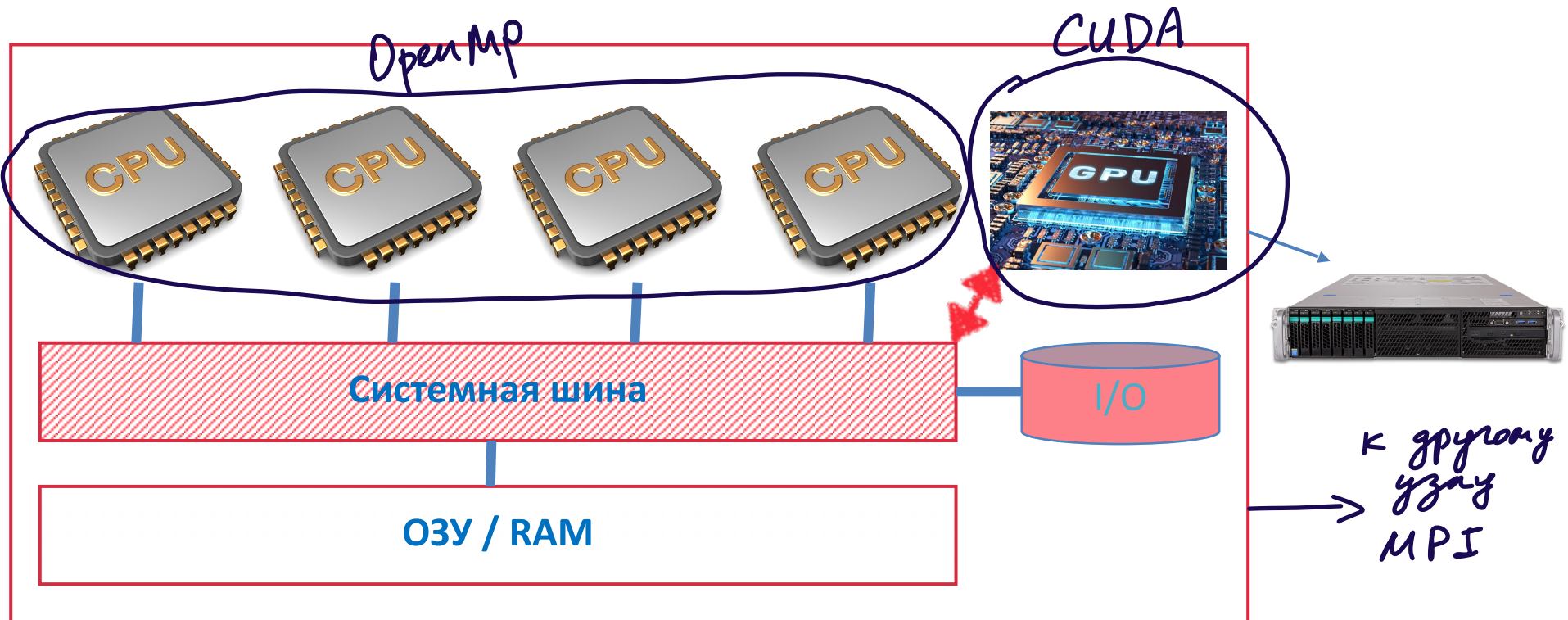


* теоретически можно и внутри узла MPI, но

Distributed memory systems + GPU

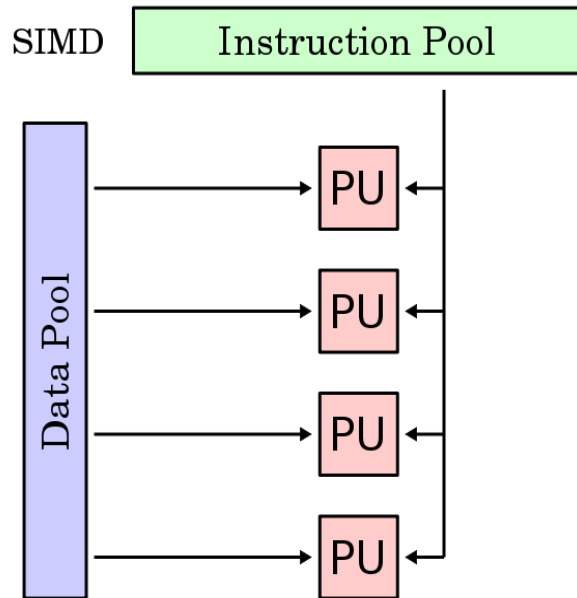


Distributed memory systems + GPU

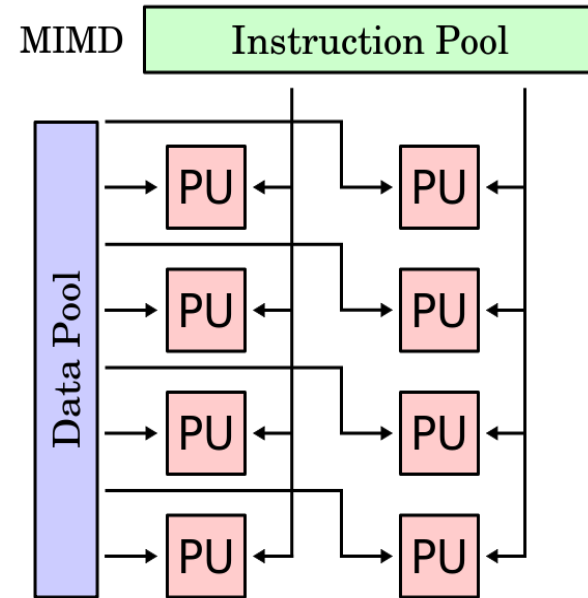


Flynn's taxonomy

GPU



CPU



Why GPUs?

Chickens vs oxen

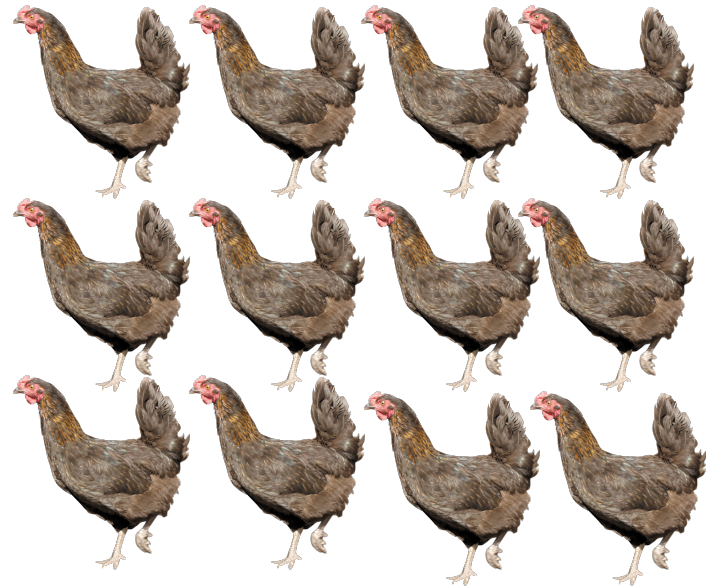


Seymour Roger Cray

"If you were plowing a field, which would you rather use: two strong oxen or 1024 chickens?"

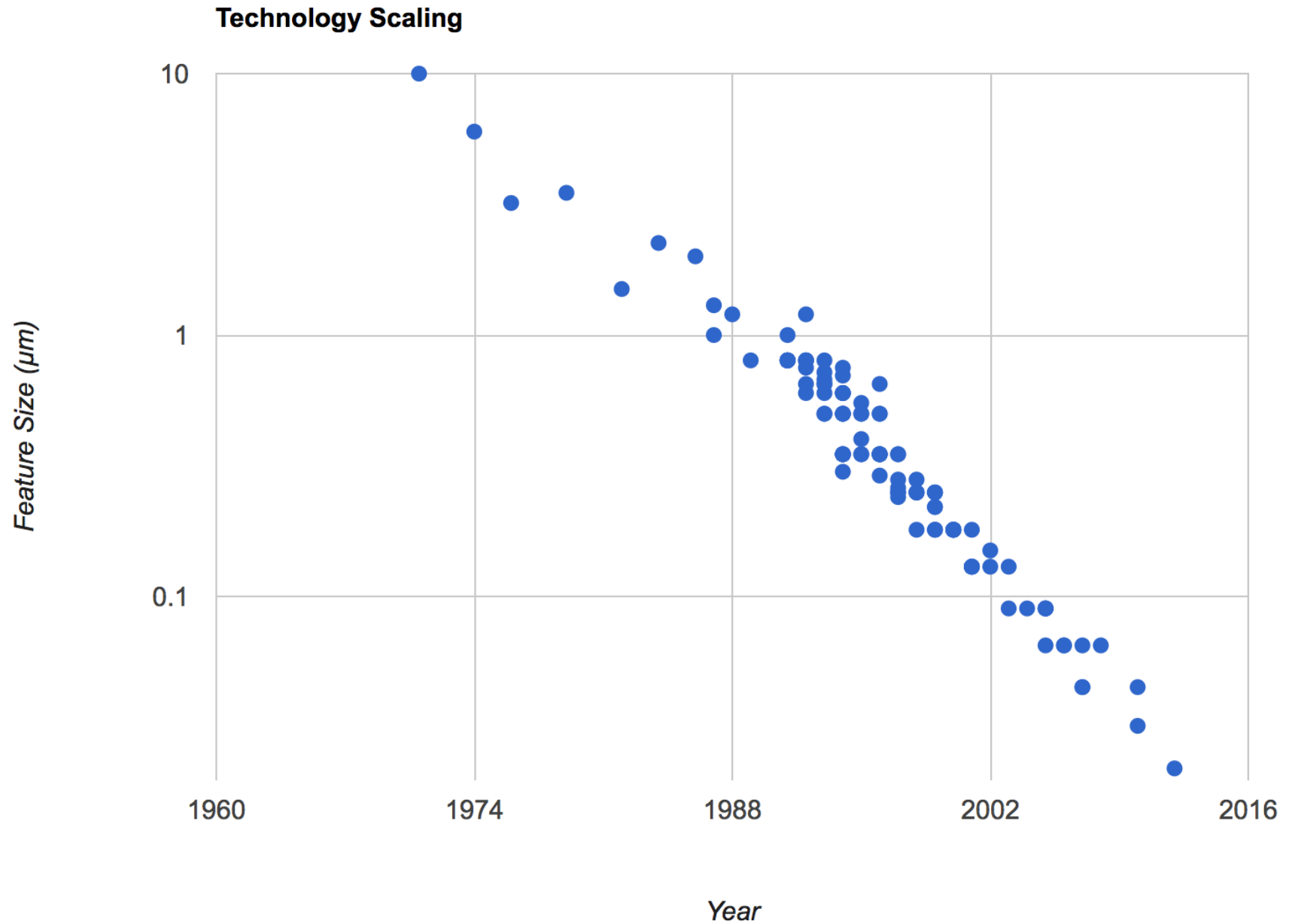


Modern CPU
~10 cores

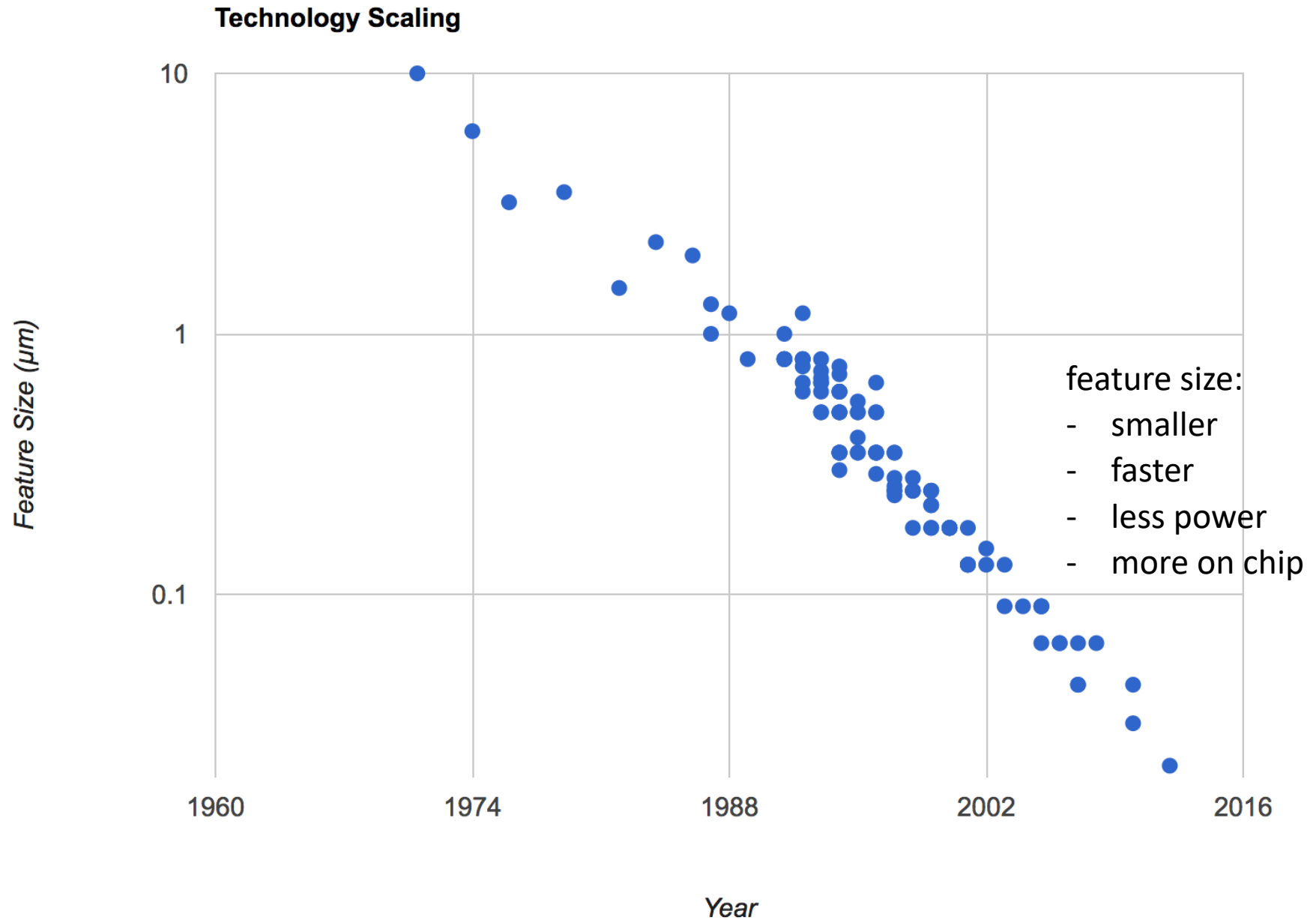


Modern GPU
~5000 cores

CPU Database (cpudb.stanford.edu)

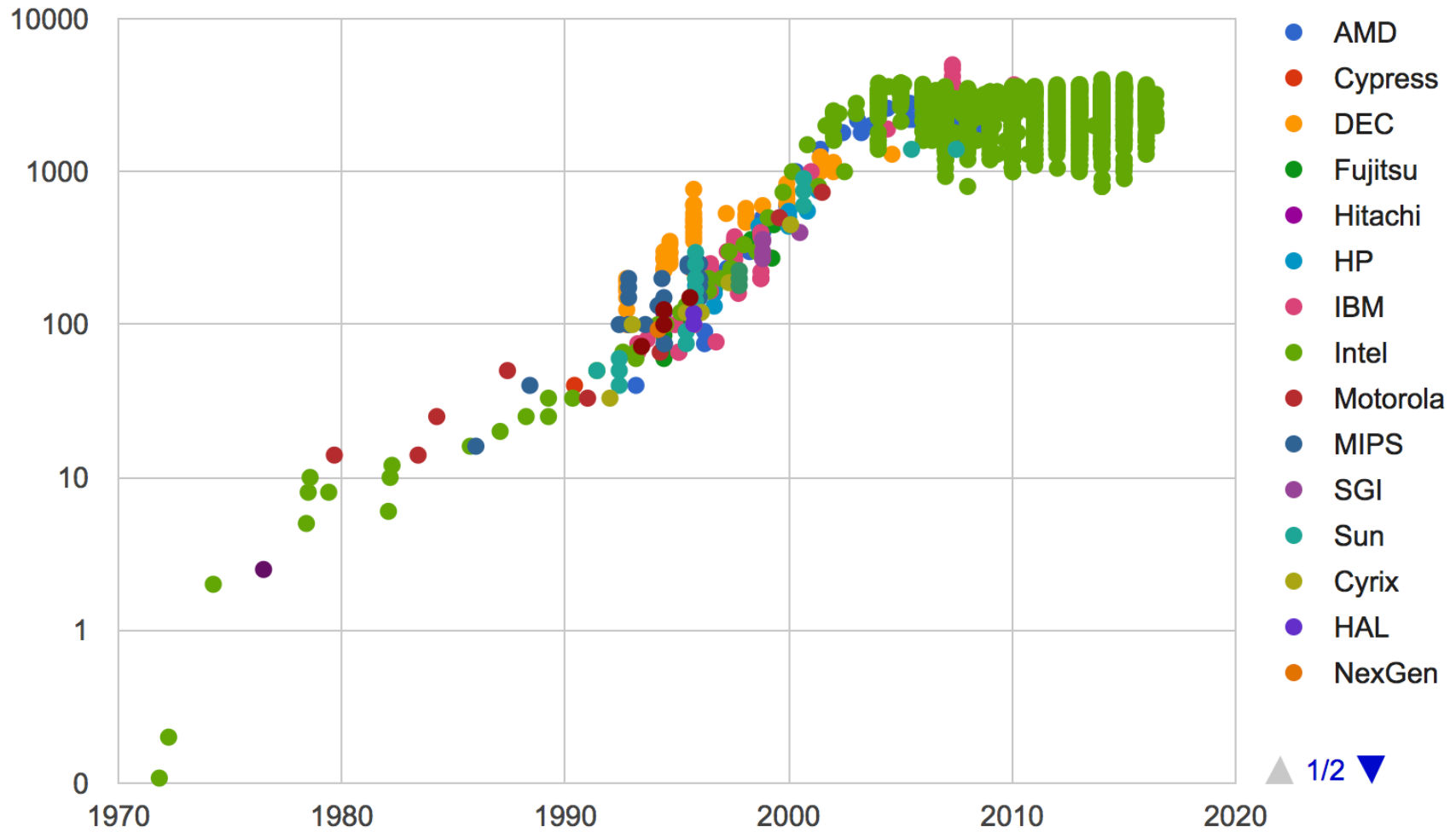


CPU Database (cpudb.stanford.edu)



CPU Database (cpudb.stanford.edu)

Clock Frequency



Can't clock further : **HEAT**

Power is the new constraint

Consequence:

- Better to build smaller more efficient processors (in terms of power)
- More of them

What processors can we build if power is the main constraint?

Modern **CPUs** are great all-around processors capable of doing all sorts of things:

- + flexibility + performance
- power consumption (expensive flops/watt)

Modern **GPUs** have simpler control structure (simpler processing units)

- + simpler processing units (many of those)
- + more power efficient (more flops/watt)
- you have to think how to program those

Latency vs Throughput

Latency
(time)
(seconds)

vs

Throughput
(tasks/time)
(flops)

Latency vs Throughput

Latency

(time)

(seconds)

vs

Throughput

(tasks/time)

(flops)

Supermarket

cash
desk



cash
desk



cash
desk



Latency vs Throughput



CPU design -- design from the point of view of customers (want to get out of supermarket as fast as possible, even though the cashier will not have any tasks to do afterwards, optimize for latency)

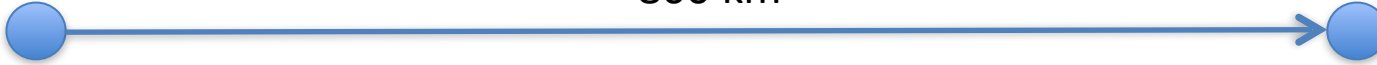
GPU design – design from the point of view of the supermarket owner (cashiers are getting paid for the whole day – they better have constant work, optimize for throughput)

Latency vs Throughput

Moscow

800 km

St. Petersburg

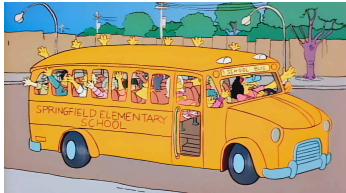


200 km/hour

2 people

Latency =

Throughput =



50 km/hour

40 people

Latency =

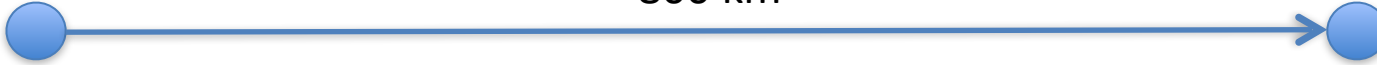
Throughput =

Latency vs Throughput

Moscow

800 km

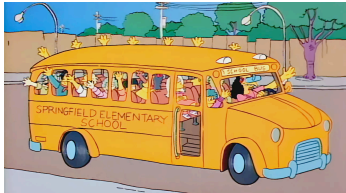
St. Petersburg



200 km/hour
2 people

Latency = 4 hours

Throughput = 0.5 people/hour



50 km/hour
40 people

Latency = 16 hours

Throughput = 2.5 people / hour

Core GPU design tenets

1. Lots of simple compute units that trade simple control for more compute;
2. Explicitly parallel programming model;
3. Optimized for throughput not latency

NVIDIA CUDA

1. CUDA is a parallel computing platform and application programming interface (API) that can be used on NVIDIA GPU cards.
2. Proprietary with closed source compilers (unlike OpenCL).

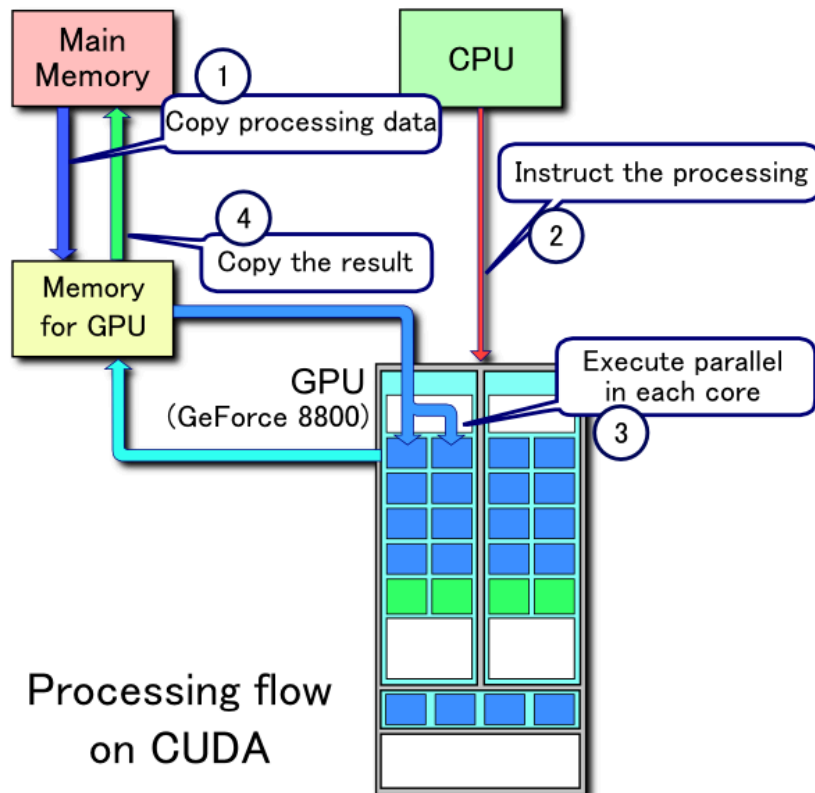
CUDA 8.0:

- CUBLAS - CUDA Basic Linear Algebra Subroutines library, see [main](#) and [docs](#)
- CUDART - CUDA RunTime library, see [docs](#)
- CUFFT - CUDA Fast Fourier Transform library, see [main](#) and [docs](#)
- CURAND - CUDA Random Number Generation library, see [main](#) and [docs](#)
- CUSOLVER - CUDA based collection of dense and sparse direct solvers, see [main](#) and [docs](#)
- CUSPARSE - CUDA Sparse Matrix library, see [main](#) and [docs](#)
- NPP - NVIDIA Performance Primitives library, see [main](#) and [docs](#)
- NVGRAPH - NVIDIA Graph Analytics library, see [main](#) and [docs](#)
- NVML - NVIDIA Management Library, see [main](#) and [docs](#)
- NVRTC - NVIDIA RunTime Compilation library for CUDA C++, see [docs](#)

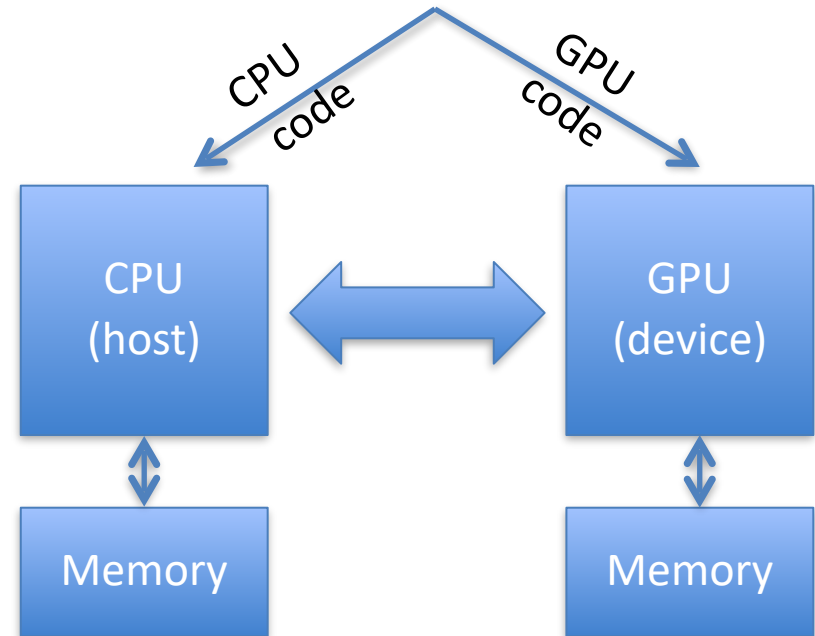
CUDA 9.0:

- CUTLASS 1.0 - custom linear algebra algorithms, see [CUDA 9.2 News](#), [Developer News](#) and [dev blog](#)

CUDA workflow and conventions



CUDA program
written in C/C++ with extension



CPU is the master and is responsible for:

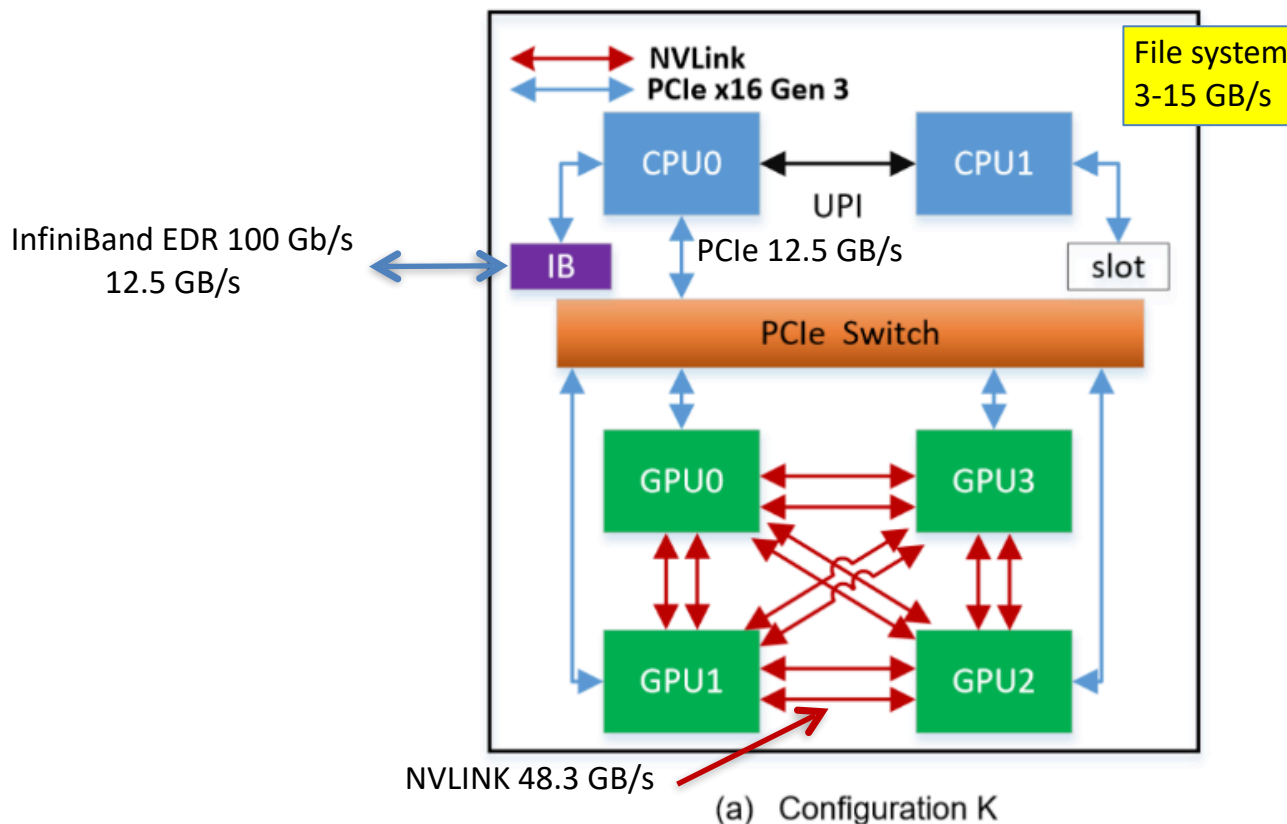
- 1) moving data from CPU mem to GPU mem
- 2) moving data from GPU mem to CPU mem
- 3) allocating memory on the GPU
- 4) launching kernels on GPU („Host launches kernels on the device“)

(1,2 – cudaMemcpy; 3 – cudaMalloc)

Typical CUDA program

1. CPU allocates and initializes storage on CPU (malloc)
2. CPU allocates storage on GPU (cudaMalloc)
3. CPU copies data to GPU (cudaMemcpy)
4. CPU launches kernels on GPU (we'll learn how)
5. CPU copies data back from GPU (cudaMemcpy)

Remember: Try to minimize copying between the CPU and GPU!!!



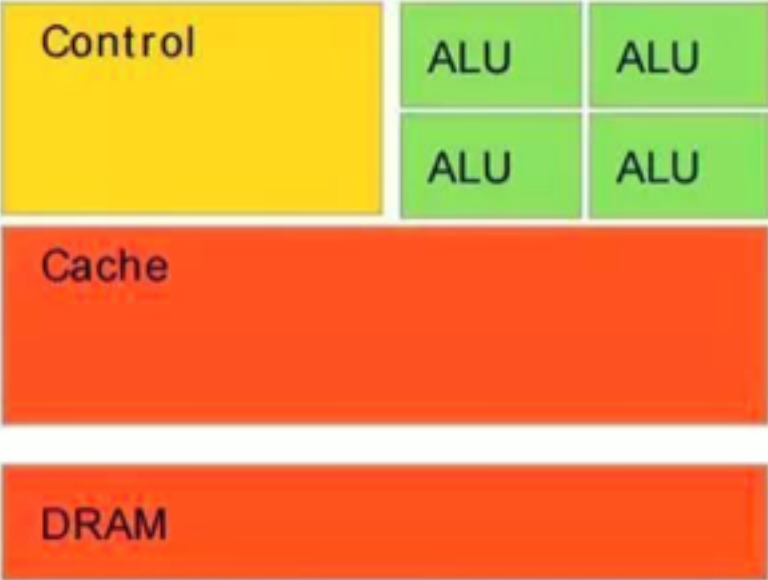
Core concept of CUDA

Come back to this slide after doing examples: make sure you understand this

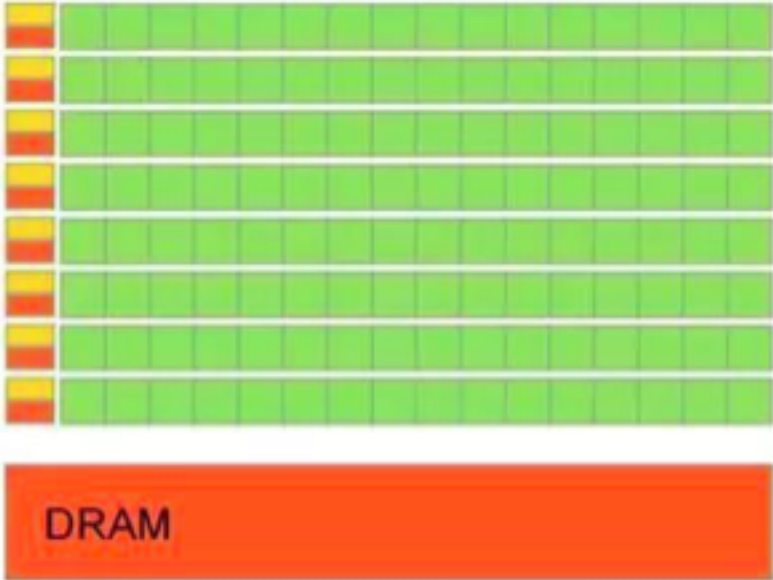
1. Kernels look like serial programs
2. Write your program as if it will run on one thread
3. The GPU will run that program on many threads (you tell the GPU how many threads to launch)

GPUs are good at:

1. Efficiently launching lots of threads (you can tell the GPU to run 10000000 of threads)
2. Running lots of threads in parallel (GPU has a maximum of threads it can run at once)



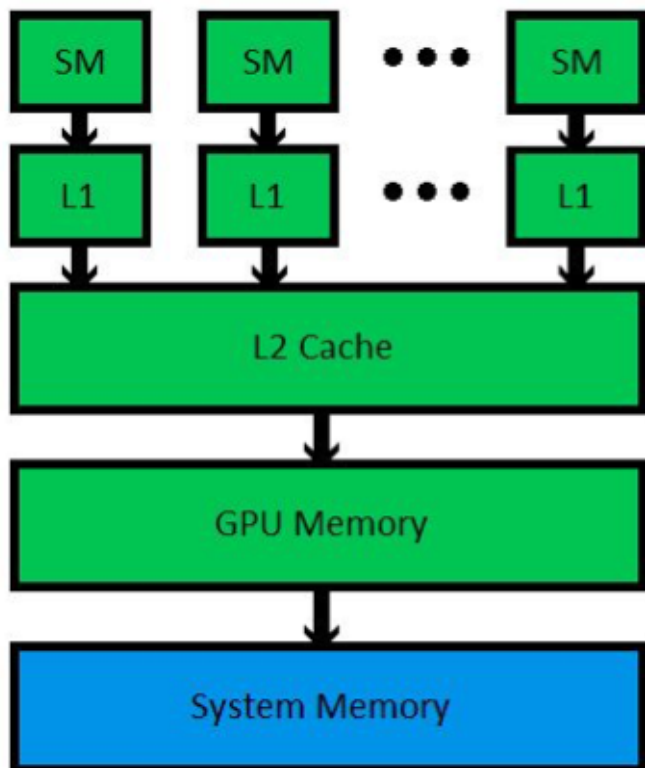
CPU



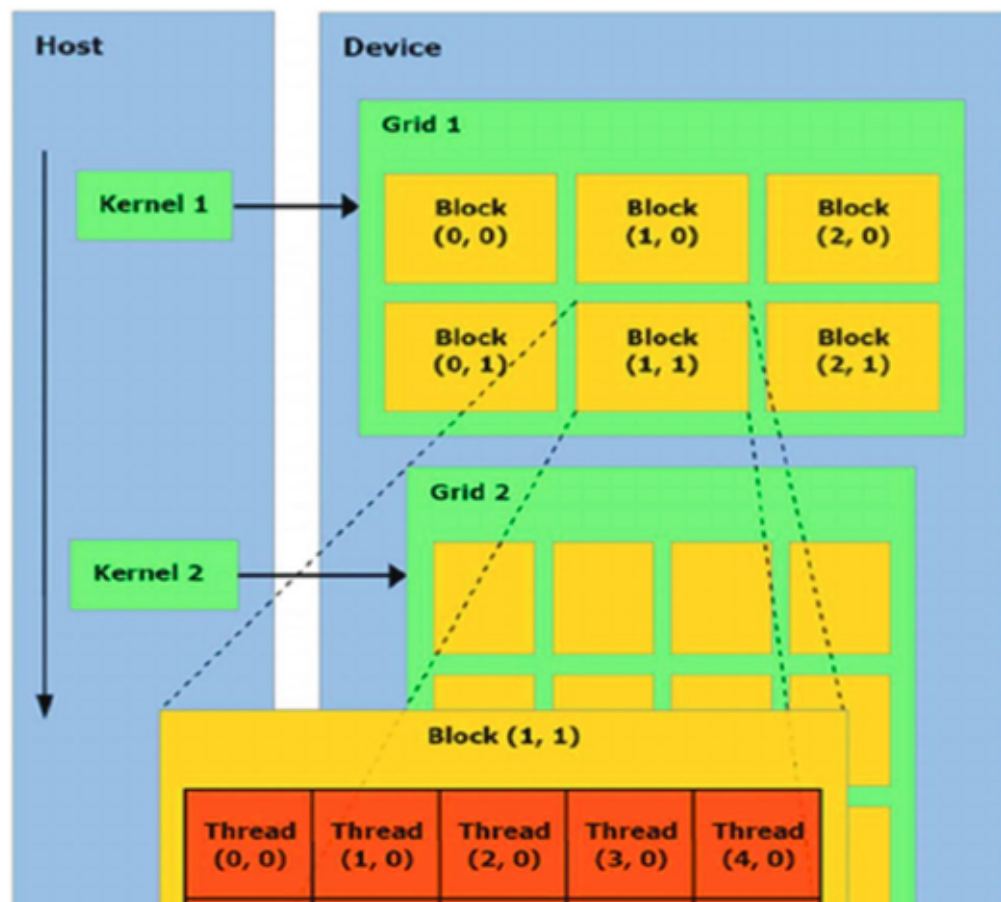
GPU

What is GPU?

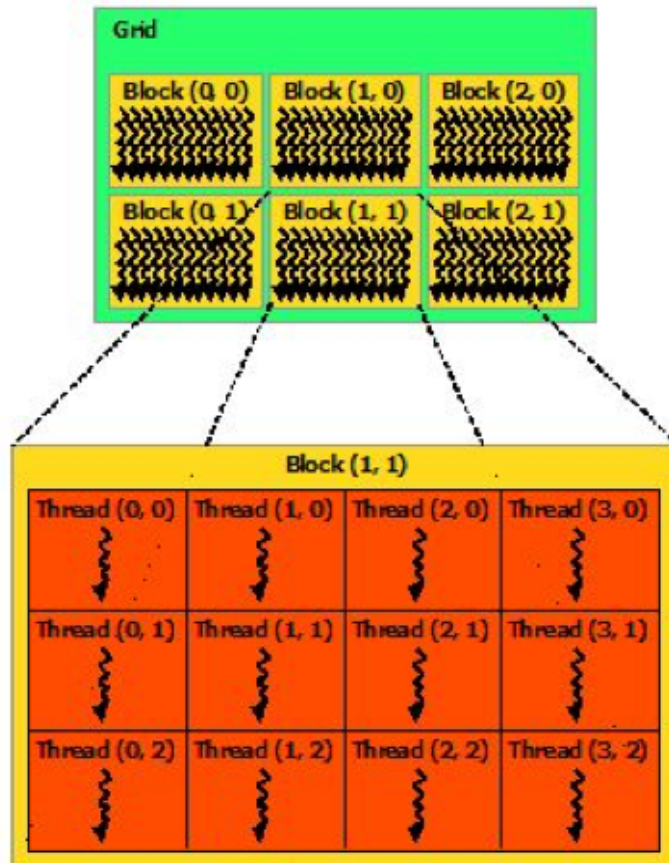
Physically



Logically

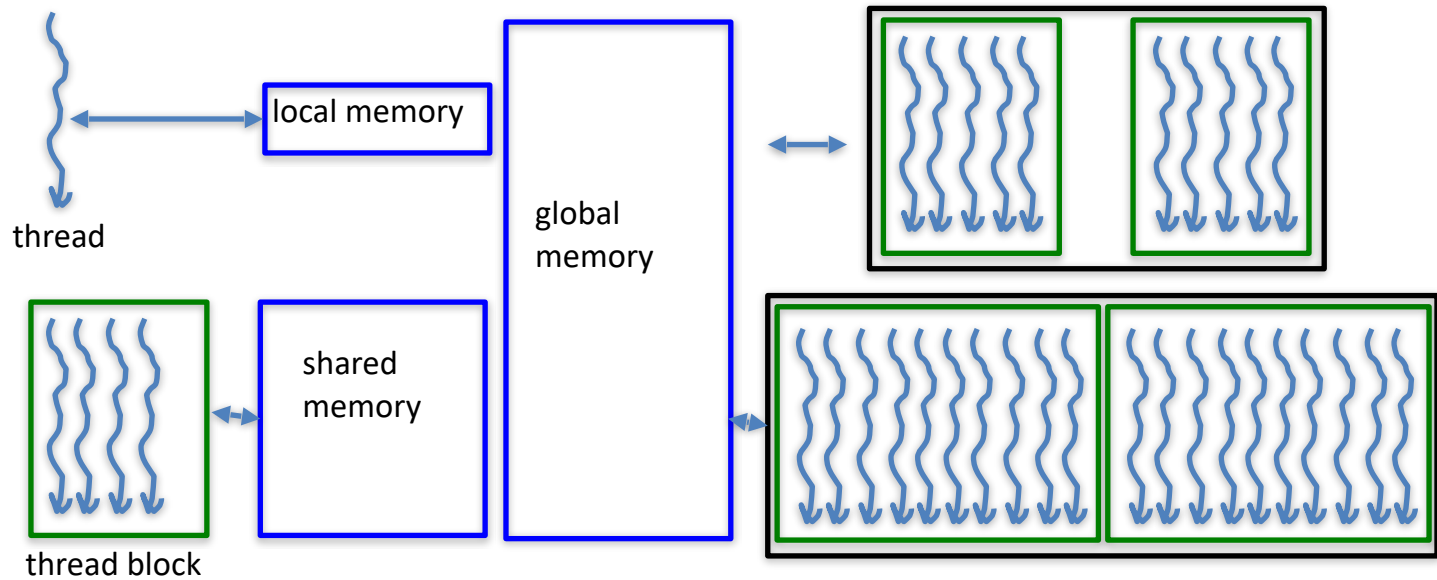


Compute Unified Device Architecture (CUDA)

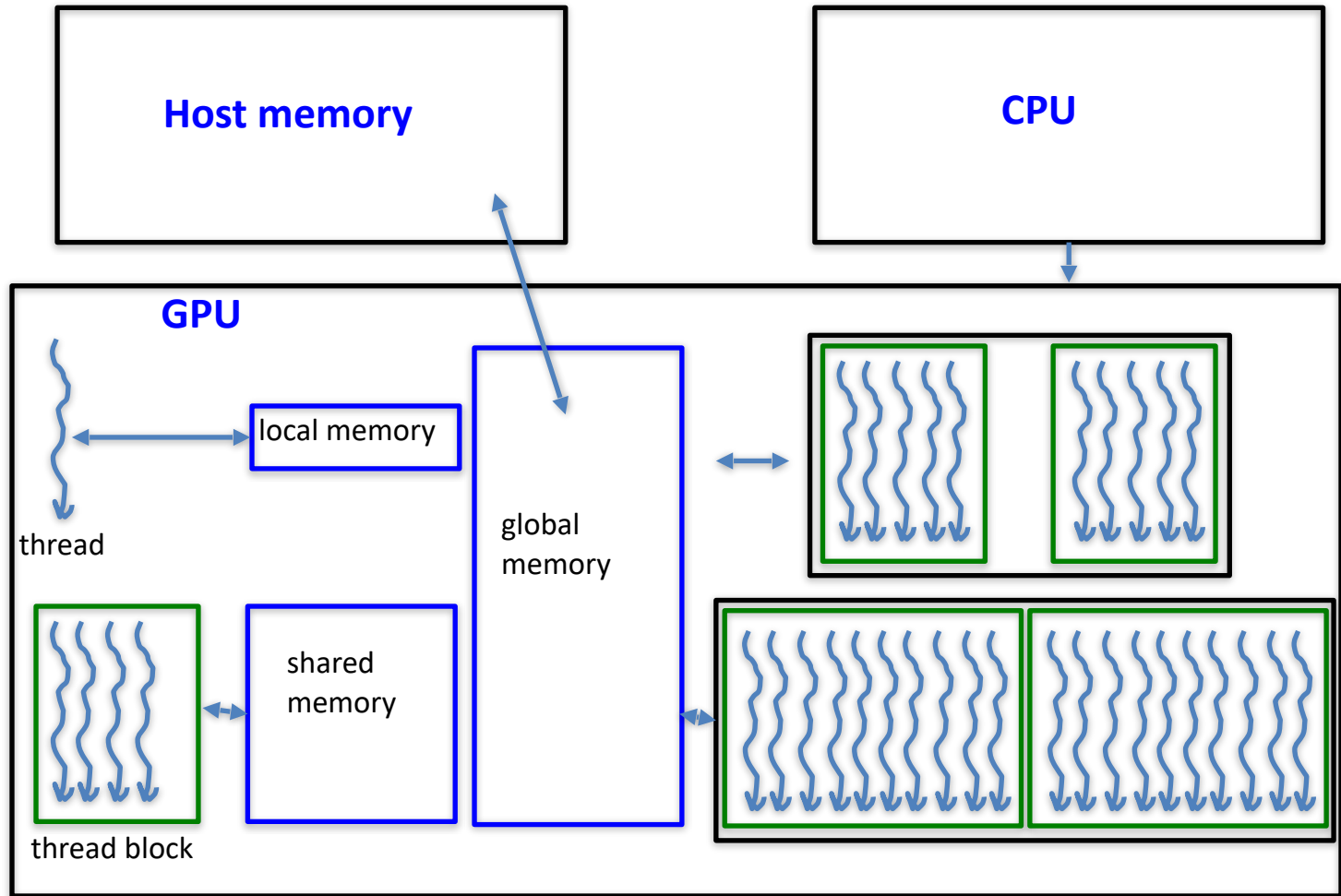


- Kernel is a serial program for one thread
- Each thread executes a kernel
- You can tell the GPU how many threads to launch
- Threads are grouped in blocks, blocks -- in grids

Memory model



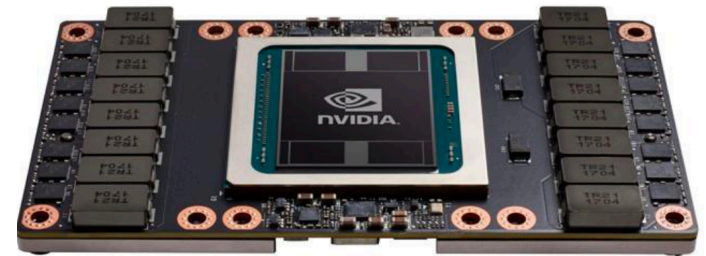
Memory model



NVidia V100 GPU (on Zhores cluster)

- ▶ 84 Volta SMs
Each SM has:
 - 64 FP32 cores
 - 64 INT32 cores
 - 32 FP64 cores
 - 8 Tensor Cores
 - Four texture units
- ▶ Eight 512-bit memory controllers (4096 bits total)

With 84 SMs, a full GV100 GPU has a total of 5376 FP32 cores, 5376 INT32 cores, 2688 FP64 cores, 672 Tensor Cores, and 336 texture units. Each HBM2 DRAM stack is controlled by a pair of memory controllers. The full GV100 GPU includes a total of 6144 KB of L2 cache. Figure 4 shows a full GV100 GPU with 84 SMs (different products can use different configurations of GV100). The Tesla V100 accelerator uses 80 SMs. Table 1 compares NVIDIA Tesla GPUs over the past five years.



Сравнение разных GPU от NVidia

Table 1. Comparison of NVIDIA Tesla GPUs

Tesla Product	Tesla K40	Tesla M40	Tesla P100	Tesla V100
GPU	GK180 (Kepler)	GM200 (Maxwell)	GP100 (Pascal)	GV100 (Volta)
SMs	15	24	56	80
TPCs	15	24	28	40
FP32 Cores / SM	192	128	64	64
FP32 Cores / GPU	2880	3072	3584	5120
FP64 Cores / SM	64	4	32	32
FP64 Cores / GPU	960	96	1792	2560
Tensor Cores / SM	NA	NA	NA	8
Tensor Cores / GPU	NA	NA	NA	640
GPU Boost Clock	810/875 MHz	1114 MHz	1480 MHz	1530 MHz
Peak FP32 TFLOPS ¹	5	6.8	10.6	15.7
Peak FP64 TFLOPS ¹	1.7	.21	5.3	7.8
Peak Tensor TFLOPS ¹	NA	NA	NA	125
Texture Units	240	192	224	320
Memory Interface	384-bit GDDR5	384-bit GDDR5	4096-bit HBM2	4096-bit HBM2
Memory Size	Up to 12 GB	Up to 24 GB	16 GB	16 GB
L2 Cache Size	1536 KB	3072 KB	4096 KB	6144 KB
Shared Memory Size / SM	16 KB/32 KB/48 KB	96 KB	64 KB	Configurable up to 96 KB
Register File Size / SM	256 KB	256 KB	256 KB	256KB
Register File Size / GPU	3840 KB	6144 KB	14336 KB	20480 KB
TDP	235 Watts	250 Watts	300 Watts	300 Watts
Transistors	7.1 billion	8 billion	15.3 billion	21.1 billion
GPU Die Size	551 mm ²	601 mm ²	610 mm ²	815 mm ²
Manufacturing Process	28 nm	28 nm	16 nm FinFET+	12 nm FFN

¹ Peak TFLOPS rates are based on GPU Boost Clock

Writing efficient programs

High-level strategies:

- 1. Maximize arithmetic intensity: math/
memory**
 - 1. maximize compute operations per thread**
 - 2. minimize memory access time**

Minimize time spent on memory

1. Move frequently accessed data to fast memory

local faster than **shared** much faster than **global** much faster than **CPU RAM**

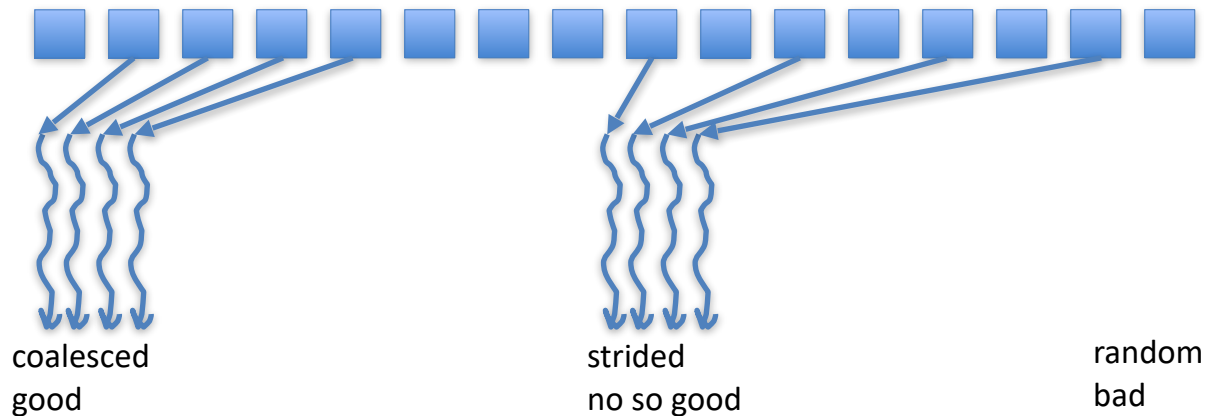
code snippet

Minimize time spent on memory

1. Move frequently accessed data to fast memory

local faster than **shared** much faster than **global** much faster than **CPU RAM**

2. Coalesce global memory access

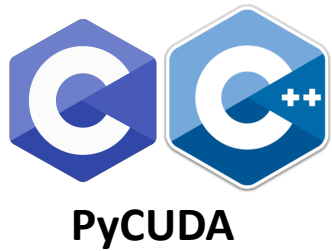


Writing efficient programs

High-level strategies:

- 1. Maximize arithmetic intensity: math/
memory**
 - 1. maximize compute operations per thread**
 - 2. minimize memory access time**
- 2. Be careful with atomics**
- 3. Avoid thread divergence**

GPUs and CUDA in Python



Low level

High level

CUDA + Python (CuPy)

CuPy is...

<https://cupy.chainer.org/>

a library to provide NumPy-compatible features with GPU



NumPy

```
import numpy as np
X_cpu = np.zeros((10,))
W_cpu = np.zeros((10, 5))
y_cpu = np.dot(x_cpu, W_cpu)
```



CuPy

```
import cupy as cp
x_gpu = cp.zeros((10,))
W_gpu = cp.zeros((10, 5))
y_gpu = cp.dot(x_gpu, W_gpu)
```

`y_cpu = cp.asnumpy(y_gpu)`

`y_gpu = cp.asarray(y_cpu)`

CUDA + Python (CuPy)

- Elementwise kernels
- Reduction kernels
- “Raw” kernels (to use this -- a dive into C required)

More Info: [**https://tinyurl.com/u8xqo9**](https://tinyurl.com/u8xqo9)
I

PyCUDA

```
import pycuda.autoinit
import pycuda.driver as drv
import numpy

from pycuda.compiler import SourceModule
mod = SourceModule("""
__global__ void multiply_them(float *dest, float *a, float *b)
{
    const int i = threadIdx.x;
    dest[i] = a[i] * b[i];
}
""")

multiply_them = mod.get_function("multiply_them")

a = numpy.random.randn(400).astype(numpy.float32)
b = numpy.random.randn(400).astype(numpy.float32)

dest = numpy.zeros_like(a)
multiply_them(
    drv.Out(dest), drv.In(a), drv.In(b),
    block=(400,1,1), grid=(1,1))

print(dest-a*b)
```

Tasks

- Saxpy
- CuPy-based bifurcation map
- Histogram
- Image blur

Detailed tasks are in jupyter notebook

Task 10. Saxpy

Single precision Alpha X Plus Y (SAXPY)

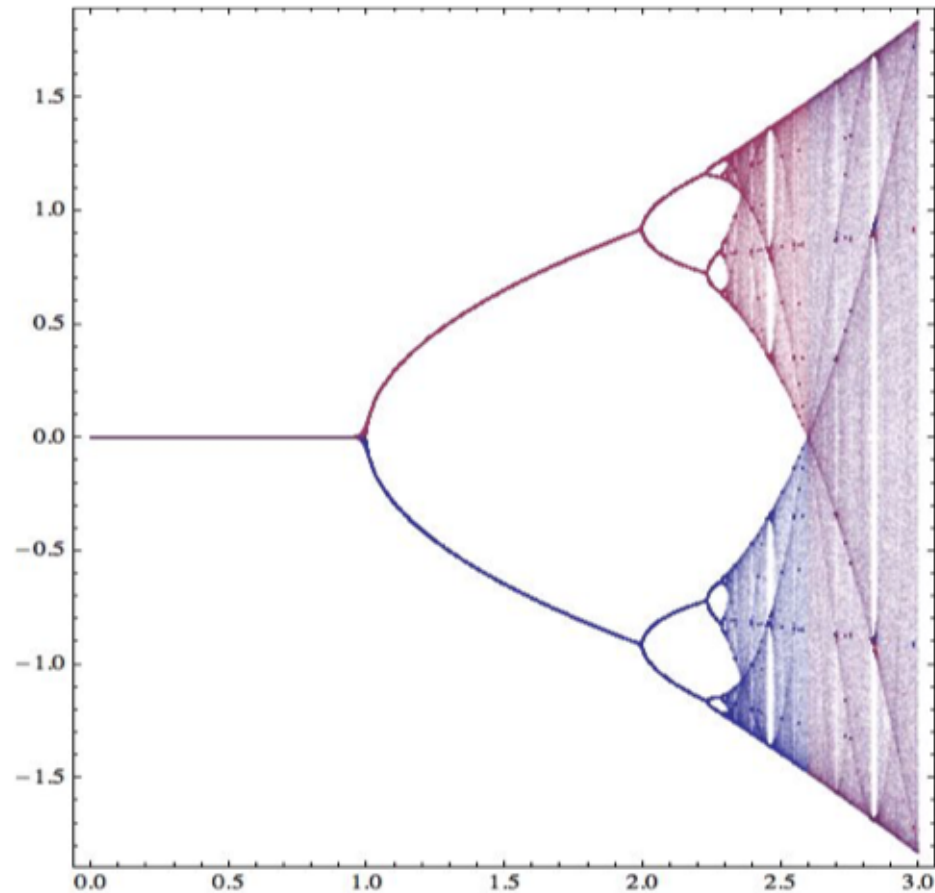
Part of Basic Linear Algebra Subroutines (BLAS) Library

$$Z = \alpha x + y$$

x, y, z : vector

α : scalar

Task 11: CuPy-based bifurcation map



Core concept of CUDA

Come back to this slide after doing examples: make sure you understand this

1. Kernels look like serial programs
2. Write your program as if it will run on one thread
3. The GPU will run that program on many threads (you tell the GPU how many threads to launch)

GPUs are good at:

1. Efficiently launching lots of threads (you can tell the GPU to run 10000000 of threads)
2. Running lots of threads in parallel (GPU has a maximum of threads it can run at once)

Simple example

In: float array [0 1 2 3 ... 63]

Out: float array [0 1 4 9 ... 63*63]

Kernel: square

```
CPU code (boring):  
for(int i=0; i<64; ++i)  
{  
    out[i]=in[i]*in[i];  
}
```

Simple example

In: float array [0 1 2 3 ... 63]

Out: float array [0 1 4 9 ... 63*63]

Kernel: square

```
CPU code (boring):  
for(int i=0; i<64; ++i)  
{  
    out[i]=in[i]*in[i];  
}
```

CPU code:

allocate memory
copy data to GPU
launch kernel



*here we explicitly
express parallelism
(tell how many threads)*

GPU code:

express out=in*in



*here we say nothing
about parallelism*

Simple example

In: float array [0 1 2 3 ... 63]

Out: float array [0 1 4 9 ... 63*63]

Kernel: square

```
CPU code (boring):  
for(int i=0; i<64; ++i)  
{  
    out[i]=in[i]*in[i];  
}
```

CPU code:

allocate memory
copy data to GPU
launch kernel



*here we explicitly
express parallelism
(tell how many threads)*

GPU code:

express out=in*in



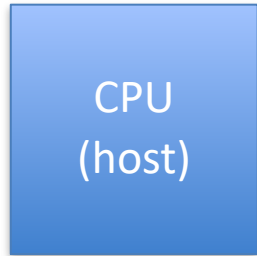
*here we say nothing
about parallelism*

```
CPU code will look like:  
squarekernel<<<64>>>(outArray, inArray)
```

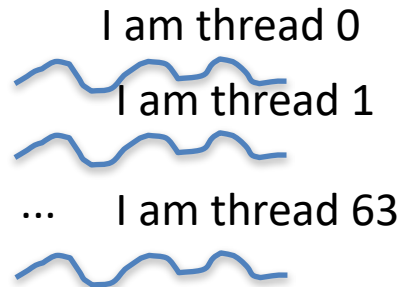
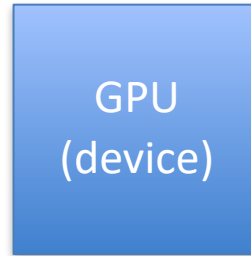
How does it work?

CPU code will look like:

```
squarekernel<<<64>>>(outArray, inArray)
```



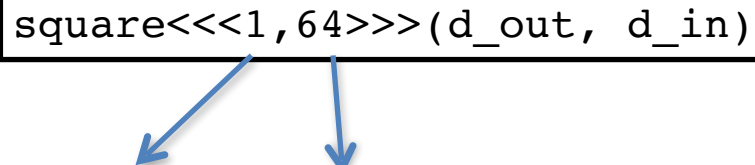
launches 64 threads on



work on item n
of the array

Configuring the kernel call

```
square<<<1,64>>>(d_out, d_in)
```



1 block of 64 threads

GPU:

1. Can run many blocks at once;
2. Maximum number of threads/block - 1024 (on most GPUs, 512 on older ones)

If we want to launch 128 threads:

```
square<<<1,128>>>(d_out, d_in)
```

If we want to launch 1280 threads:

```
square<<<10,128>>>(d_out, d_in)
```

```
square<<<5,256>>>(d_out, d_in)
```

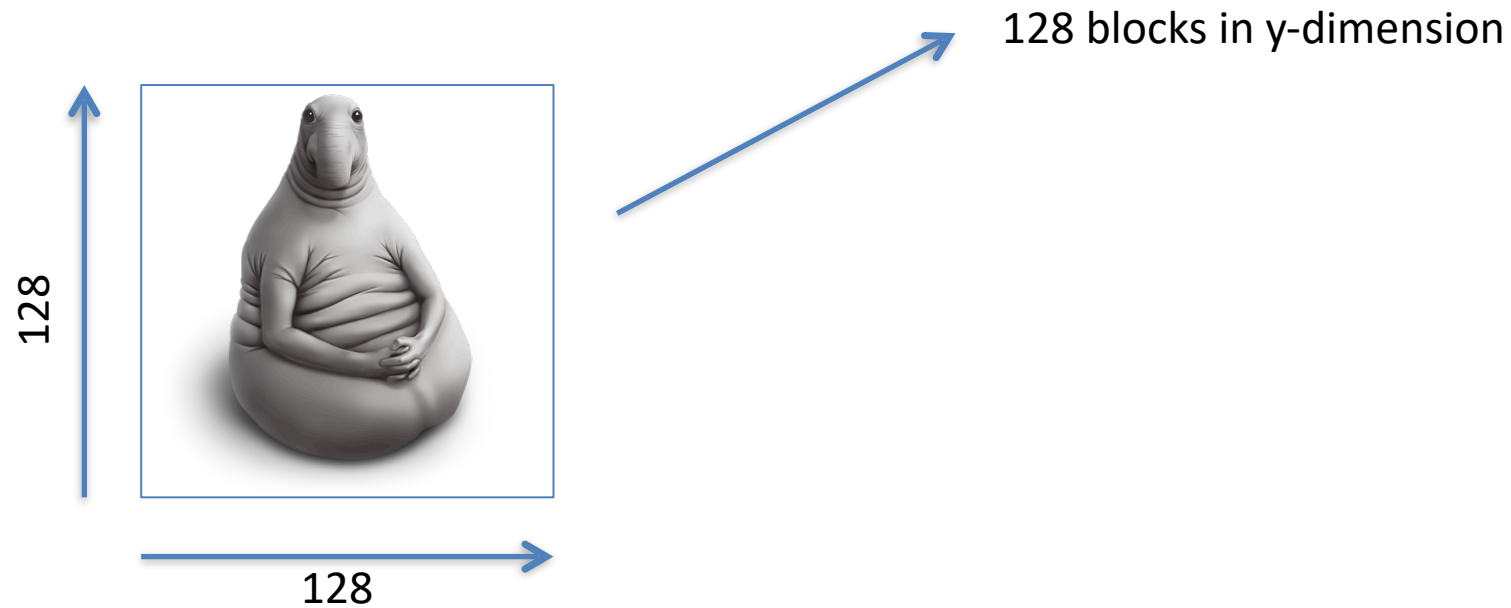
```
square<<<1,1280>>>(d_out, d_in)
```

???

Each thread knows its index within the block + index of the block. We can also do 2D and 3D mapping of blocks and threads.

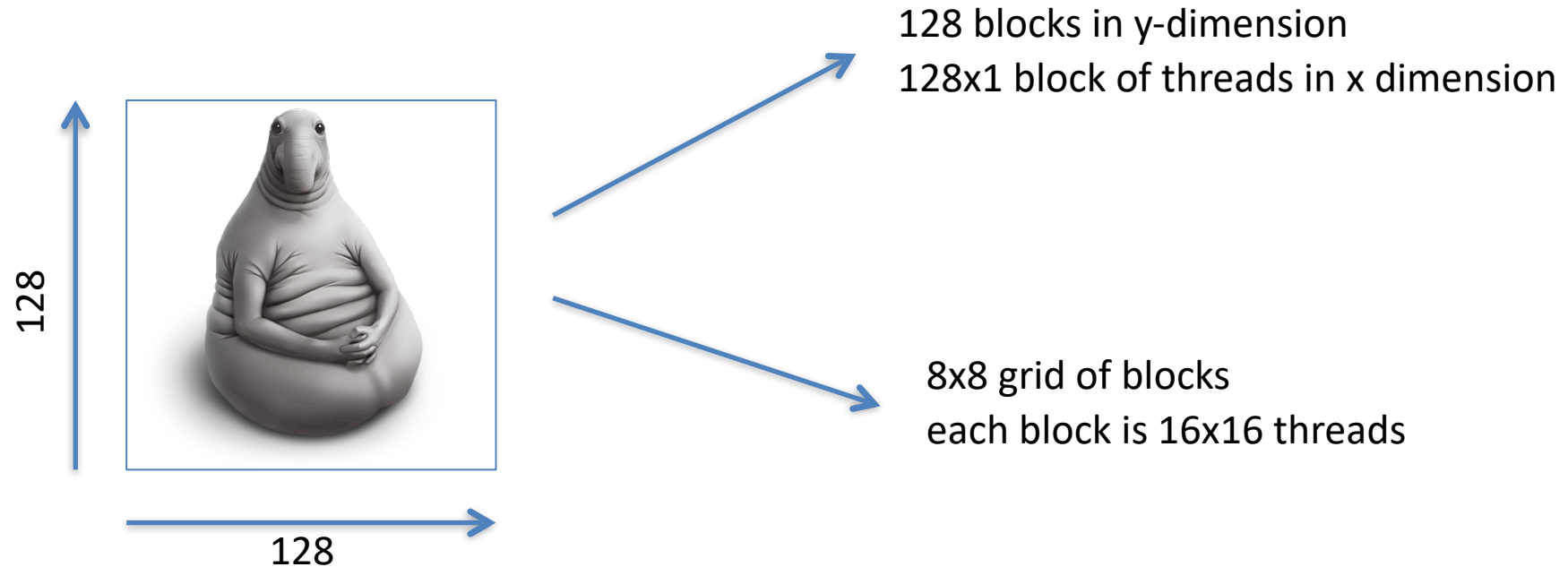
Configuring the kernel call

Image processing: one thread per pixel



Configuring the kernel call

Image processing: one thread per pixel



kernel<<<GRID OF BLOCKS, BLOCK OF THREADS>>>(...)

1d,2d or 3d

1d,2d or 3d

dim3(x,y,z)

dim3(w,1,1)=dim3(w)=w

Configuring the kernel call

General call:

```
kernel<<<dim3(bx,by,bz),dim3(tx,ty,tz)>>>(...)
```

Useful:

threadIdx.x - thread index within block

threadIdx.y

threadIdx.z

blockDim - size of a block (blockDim.x, blockDim.y, blockDim.z)

blockIdx - block within grid (blockIdx.x, blockIdx.y, blockIdx.z)

gridDim - size of a grid (gridDim.x, gridDim.y, gridDim.z)

```
kernel<<<dim3(8,4,2), dim3(16,16)>>>(...)
```

How many blocks?

How many threads per block?

Total threads?

Map parallel abstraction

MAP:

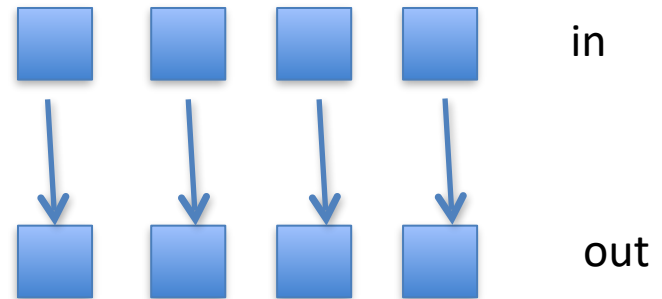
- set of elements to process [64 floats]
- function to run on each element [“square”]

MAP(elements, function)

GPUs are good at MAP:

- GPUs have many processors
- GPUs optimize for throughput

MAP communication pattern:



CUDA hello world

```
#include <stdio.h>
#include <math.h>

__global__ void Hello(void)
{
    int globalidx = threadIdx.z * blockDim.x *
blockDim.y + threadIdx.y * blockDim.x + threadIdx.x;

    printf("Hello from tx = %d\t ty=%d\t tz=%di \t
gi=%d\n", threadIdx.x, threadIdx.y,
threadIdx.z, globalidx);
}

int main(void)
{
    printf("This is done by CPU before CUDA threads\n");

    Hello<<<1,dim3(1,4,2)>>>();

    cudaDeviceSynchronize();
    return 0;
}
```




<http://tiny.cc/fyzxoz>

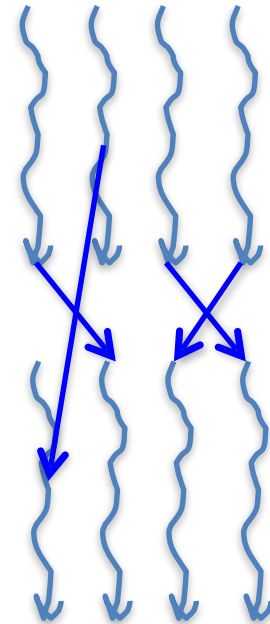
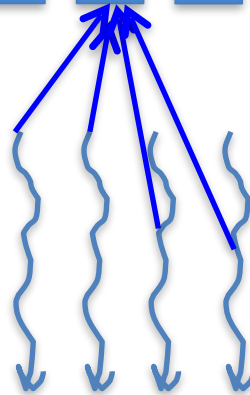
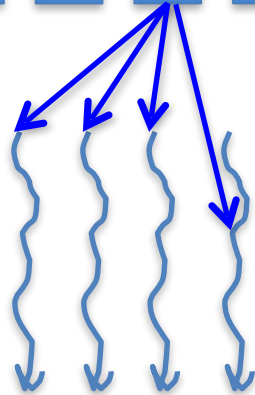
<http://tiny.cc/tzzxoz>

CUDA memory access patterns

CUDA: multiple threads working together

Communication is key!

GPU memory



threads

Example:

$C[5] = C[5] + 1$

Parallel communication patterns

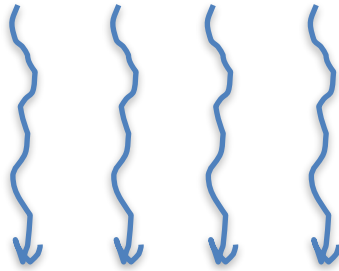
We have already learned and used parallel communication patterns in MPI.

MAP

GPU memory



$f(x)$



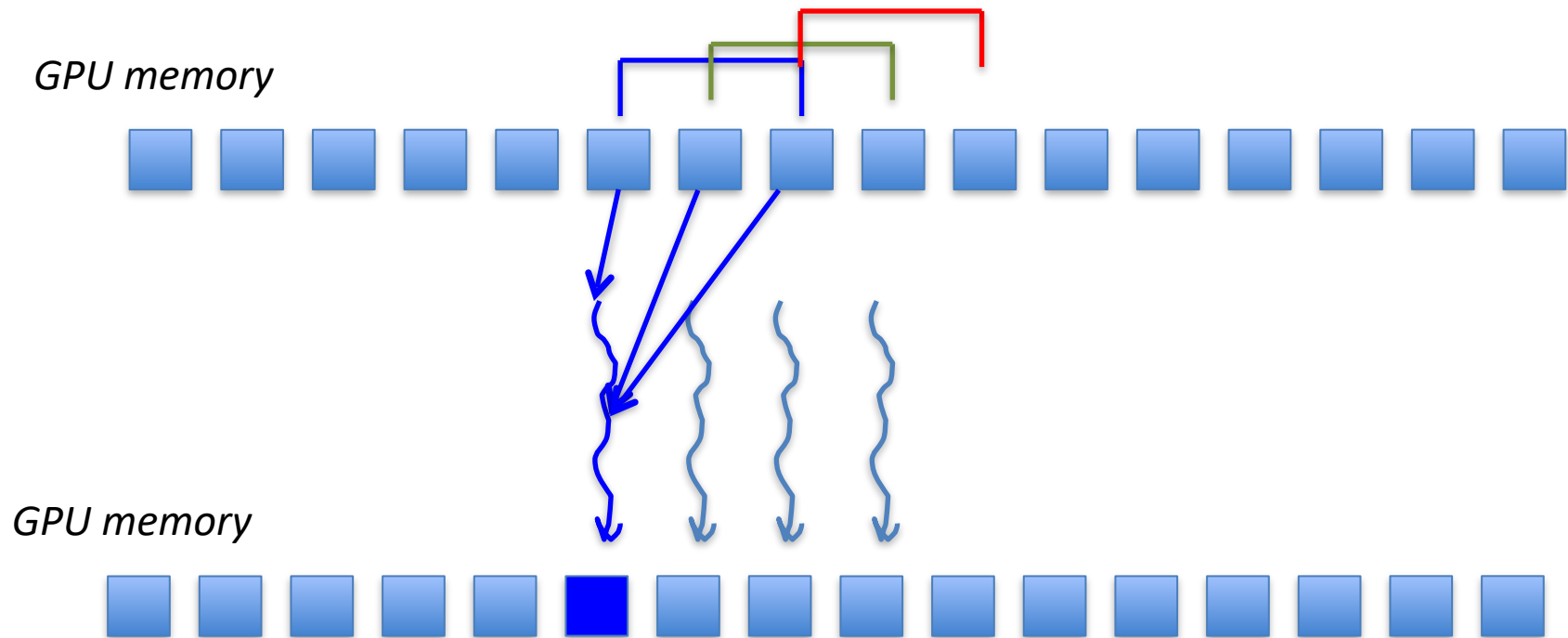
GPU memory



GPUs are really good at MAP

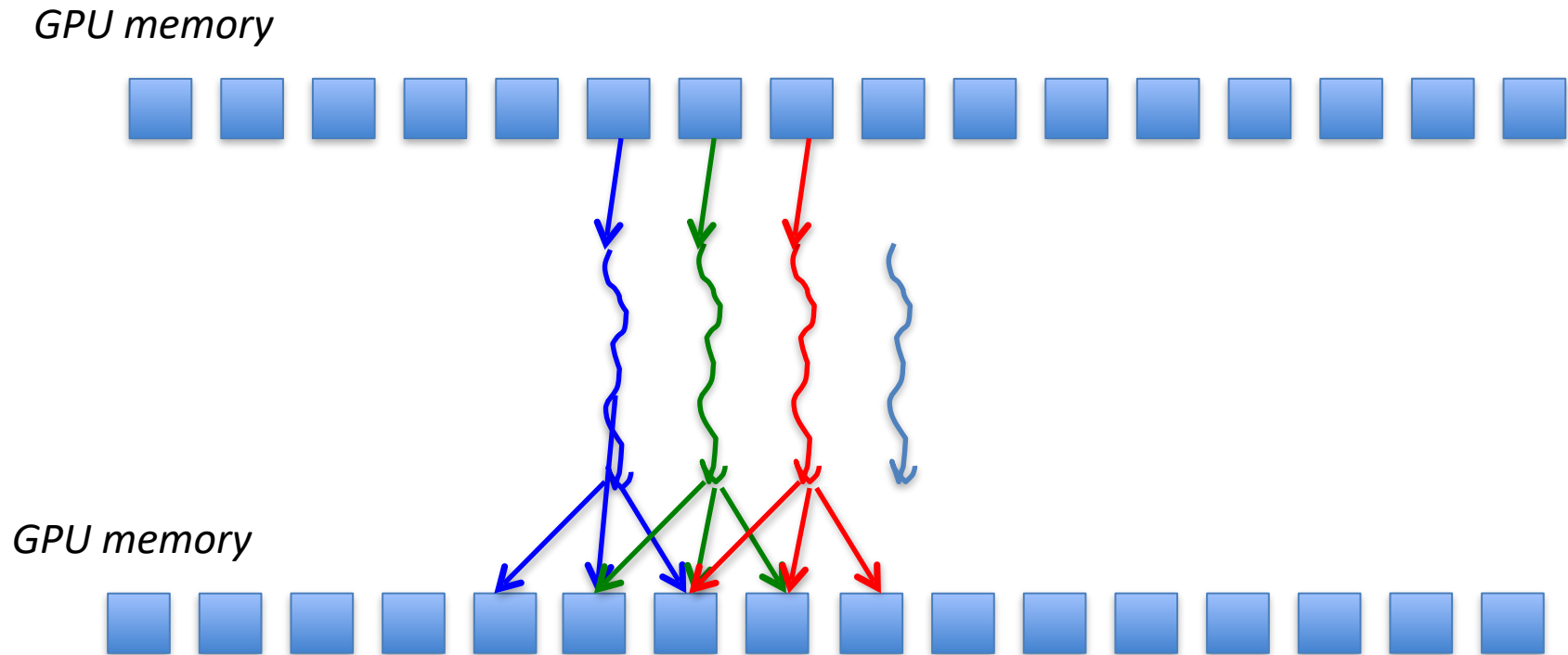
Parallel communication patterns

GATHER



Parallel communication patterns

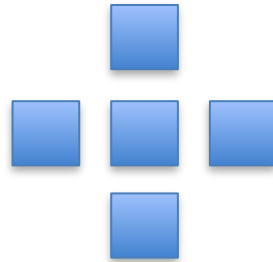
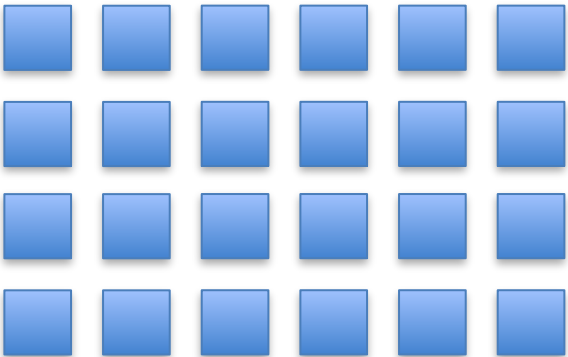
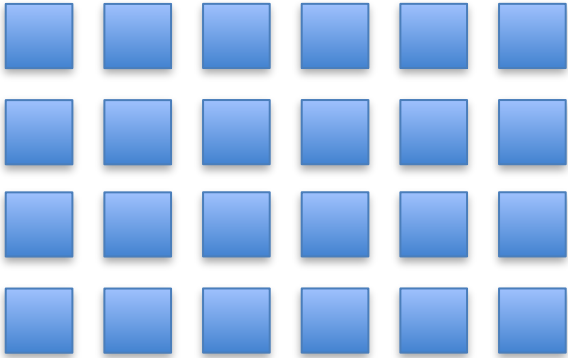
SCATTER



Parallel communication patterns

STENCIL

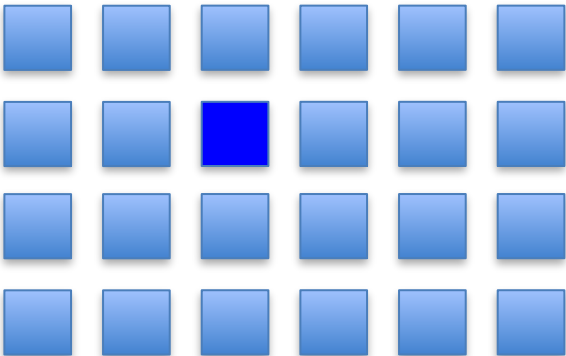
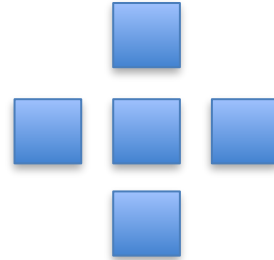
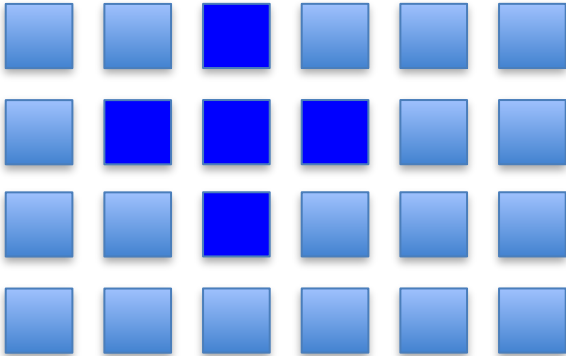
GPU memory



Parallel communication patterns

STENCIL

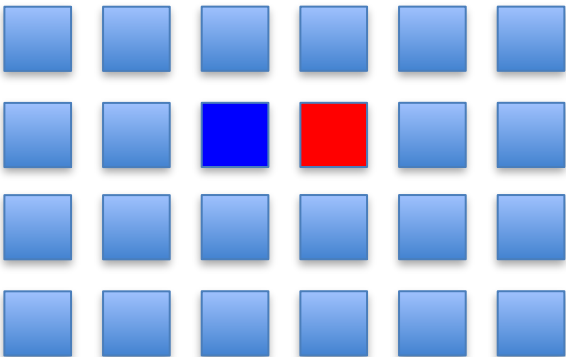
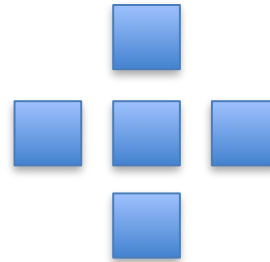
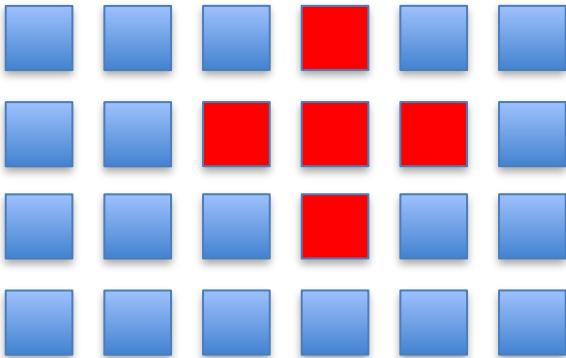
GPU memory



Parallel communication patterns

STENCIL

GPU memory

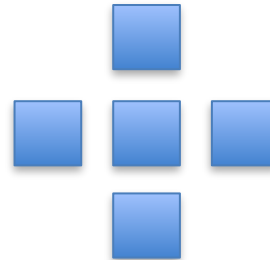
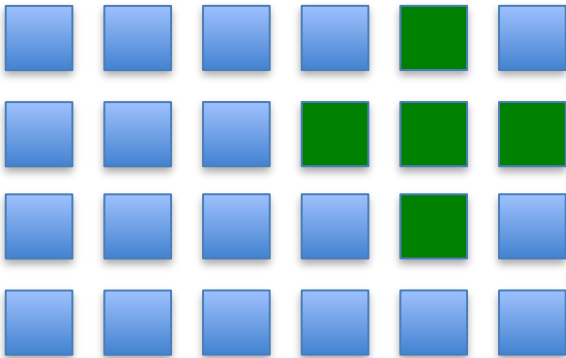


Parallel communication patterns

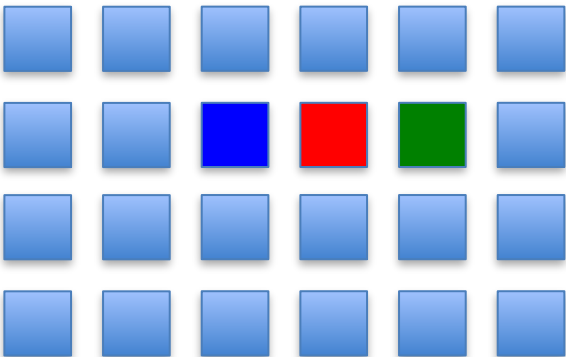
STENCIL

Data reuse!

GPU memory



2D von Neumann

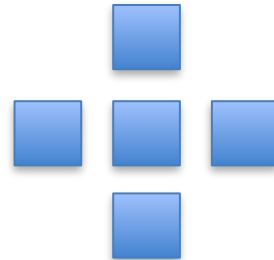
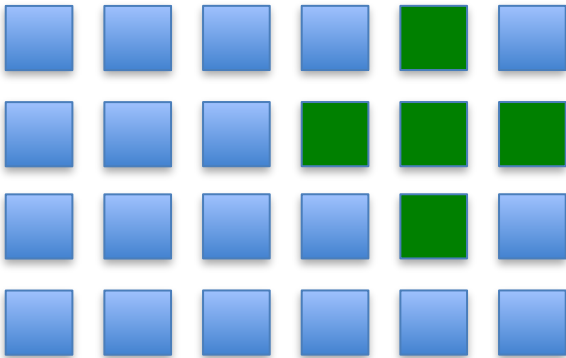


Parallel communication patterns

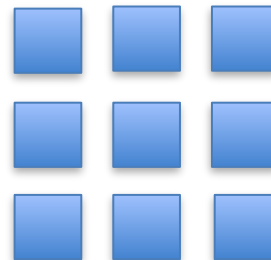
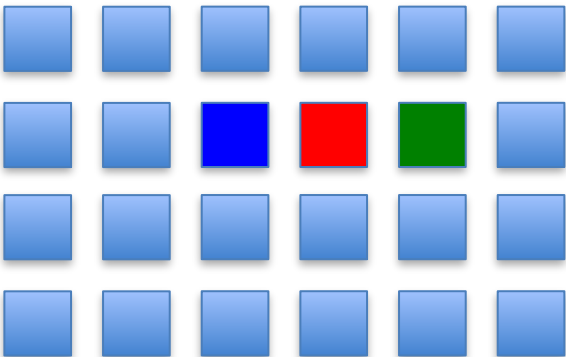
STENCIL

Data reuse!

GPU memory



2D von Neumann



2D Moore

Parallel communication patterns

TRANSPOSE

2d array

1	2	3	4
5	6	7	8

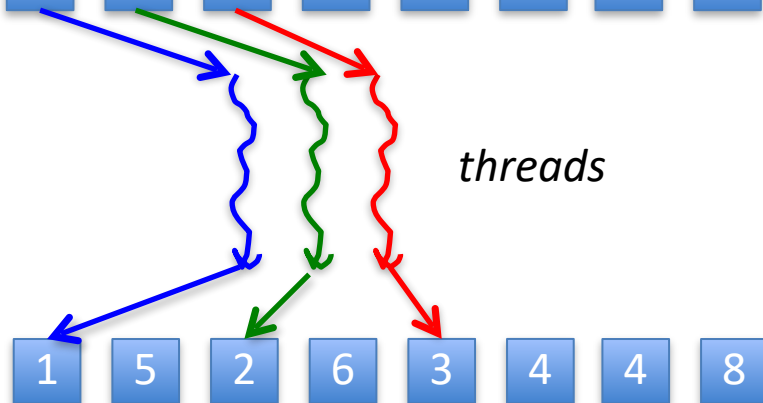


1	5
2	6
3	7
4	8



GPU memory (row-wise)

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---



Parallel communication patterns

TRANSPOSE

2d array

1	2	3	4
5	6	7	8

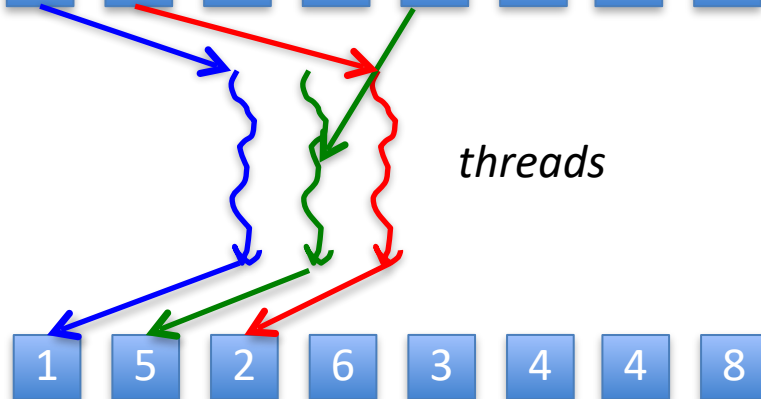


1	5
2	6
3	7
4	8



GPU memory (row-wise)

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---



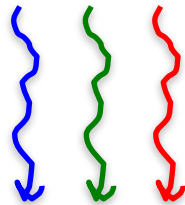
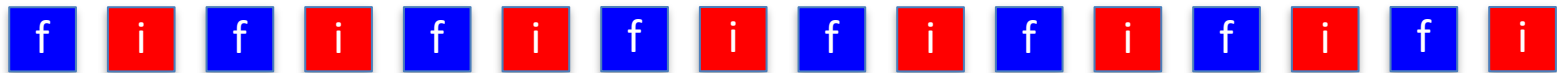
Parallel communication patterns

TRANSPOSE

```
struct foo
{
    float f;
    int i;
}
```

```
foo array[1000];
```

array of structures (AoS)



structure of arrays (SoA)



Parallel communication patterns

```
float out[], in[];  
int i = threadIdx.x;  
int j = threadIdx.y;
```

```
const float pi = 3.1415;
```

```
out[i] = pi * in[i];
```



```
out[i + j*128] = in[j + i*128];
```



```
if (i % 2) {  
    out[i-1] += pi * in[i]; out[i+1] += pi * in[i];  
  
    out[i] = (in[i] + in[i-1] + in[i+1]) * pi / 3.0f;  
}
```

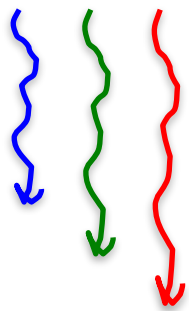
Parallel communication patterns

MAP	one-to-one operations
TRANPOSE	
GATHER	many-to-one
SCATTER	one-to-many
STENCIL	several-to-one
REDUCE	all-to-one
SCAN/SORT	all-to-all

1. How can threads efficiently access memory in concert?
2. How to exploit data reuse?
3. How can threads communicate partial results?

Summary of GPU programming model

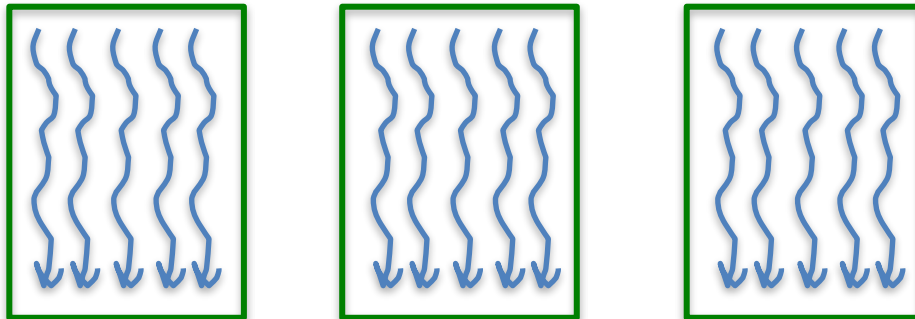
kernels - *small parts of the code that run on GPU (C/C++)*



threads

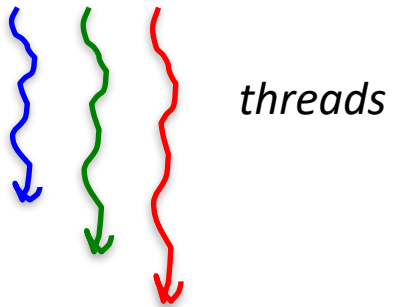
each thread might take a different path (if clauses, for loops, etc)

The key about threads is that they come in thread blocks

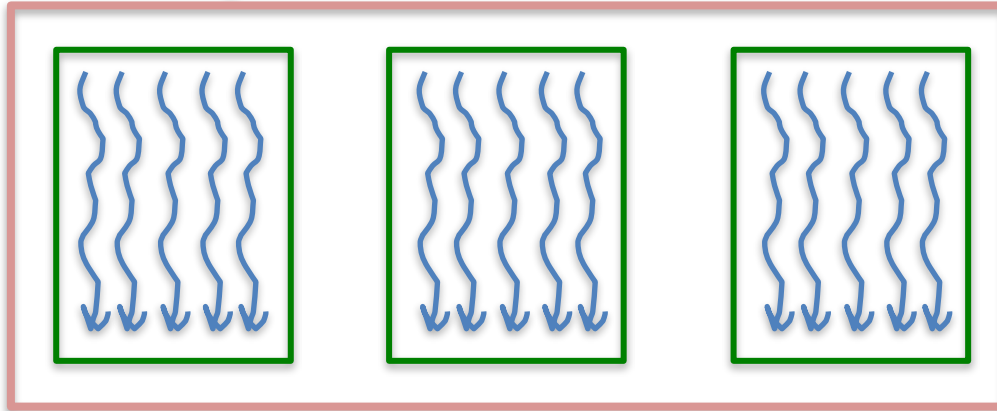


thread blocks: group of threads that cooperate to solve a certain (sub) problem

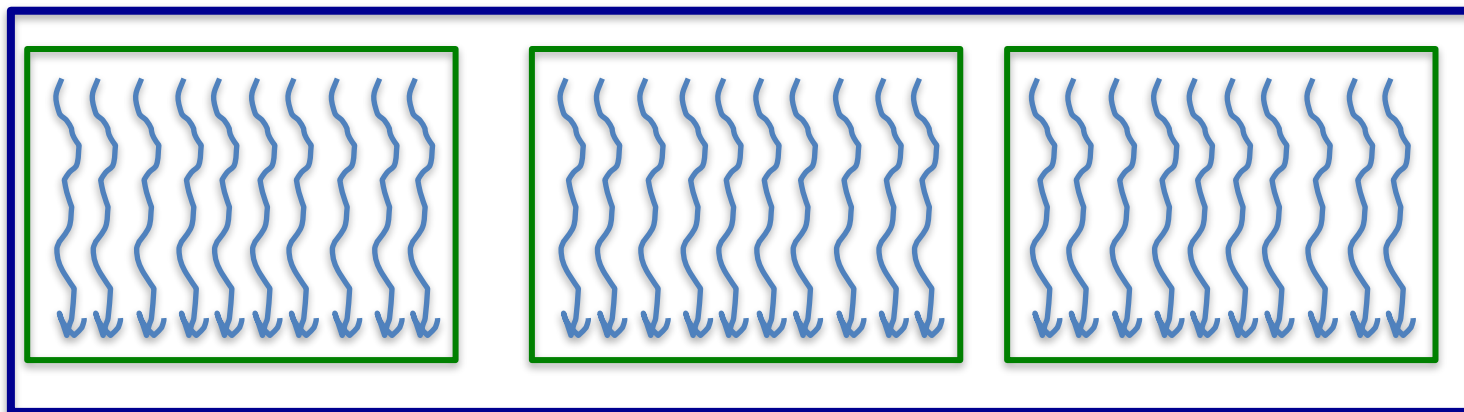
Summary of GPU programming model



thread blocks: group of threads that cooperate to solve a certain (sub) problem



kernel square



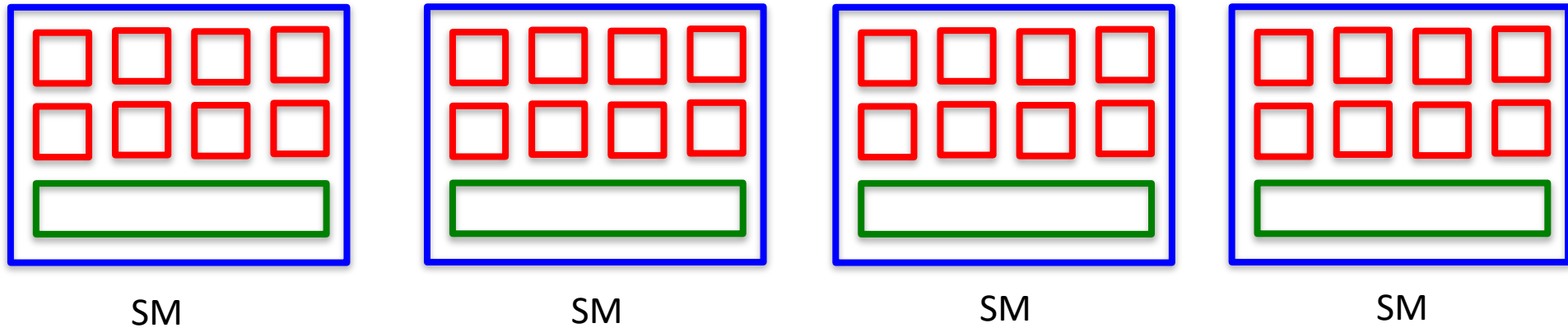
kernel sum

GPU Hardware

Questions to be answered:

1. Why do we divide problem into blocks?
2. When are blocks run?
3. In what order do they run?
4. How can threads cooperate?
5. With what limitations do threads cooperate?

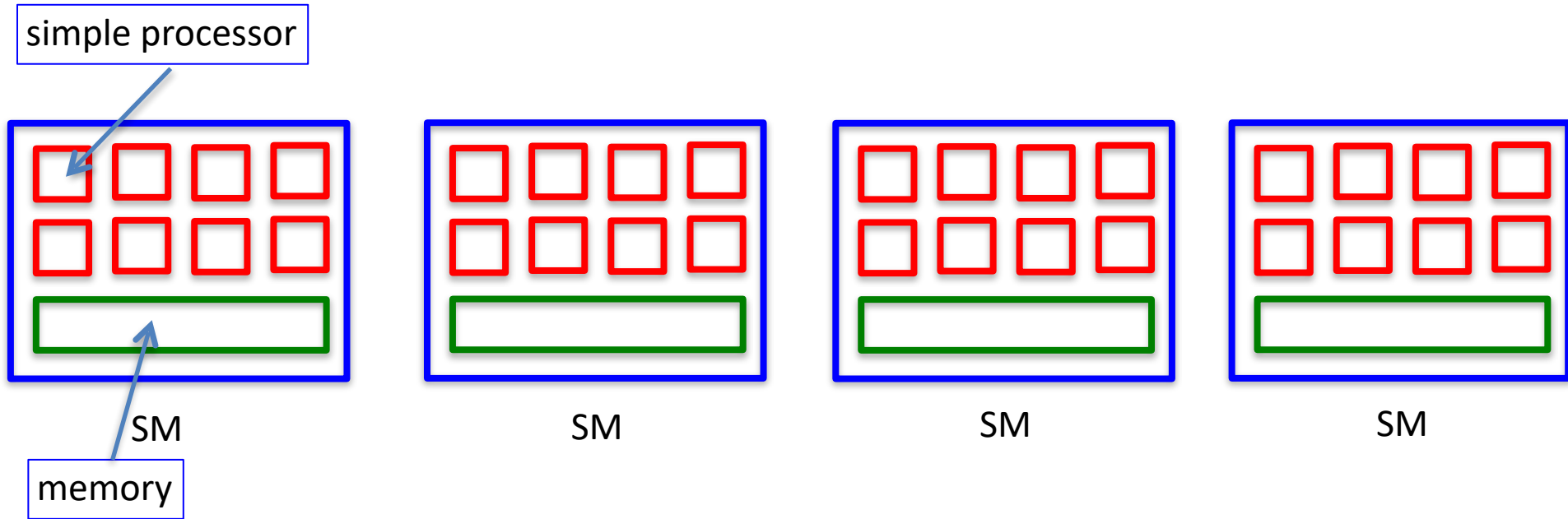
GPU Hardware



SM = streaming multiprocessor (number of those depends on a GPU)

Zhores cluster has Nvidia V100 installed; Nvidia V100 has 84 SMs

GPU Hardware



SM = streaming multiprocessor (number of those depends on a GPU)

Zhores cluster has Nvidia V100 installed; Nvidia V100 has 84 SMs

You have CUDA program with a bunch of threads organized in thread blocks:

GPU is responsible for allocating blocks to SMs!

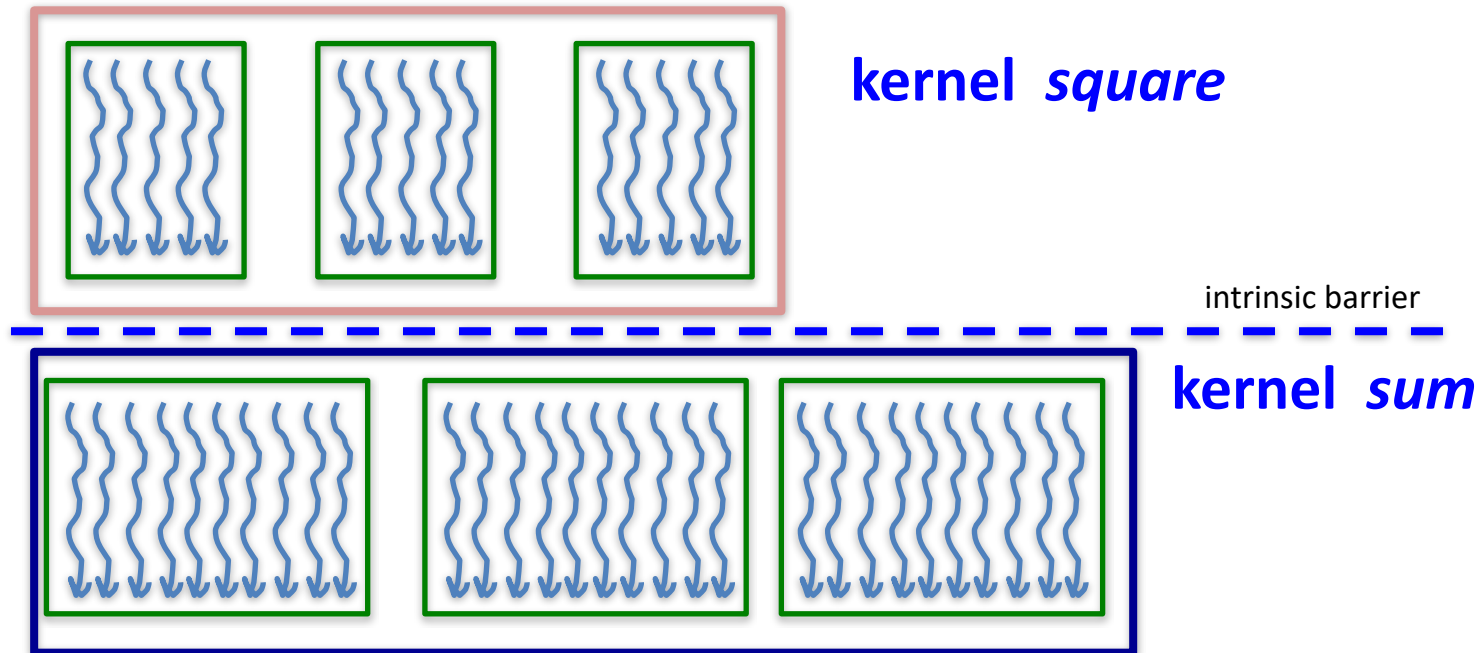
Thread block may not run on more than 1 SM!

All the SMs run in parallel and independently

GPU Hardware

CUDA guarantees:

1. All threads in a block run on the same SM at the same time.
2. All blocks in a kernel finish before any blocks from the next kernel run.



GPU Hardware

Question:

Given a single kernel that is launched on many thread blocks including X and Y, the programmer can specify:

1. That block X will run at the same time as block Y
2. That block X will run after block Y
3. That block X will run on SM 10

GPU Hardware

CUDA makes a few guarantees about when and where thread blocks can run

Advantages:

- efficiency
- scalability

Consequences:

- no assumptions on what blocks run on a given SM
- no explicit communication between blocks

GPU Hardware

```
#include <stdio.h>

__global__ void hello()
{
    printf("Hello world from thread in a block %d\n", blockIdx.x);
}

int main(int argc, char **argv)
{
    hello<<<8, 1>>>();

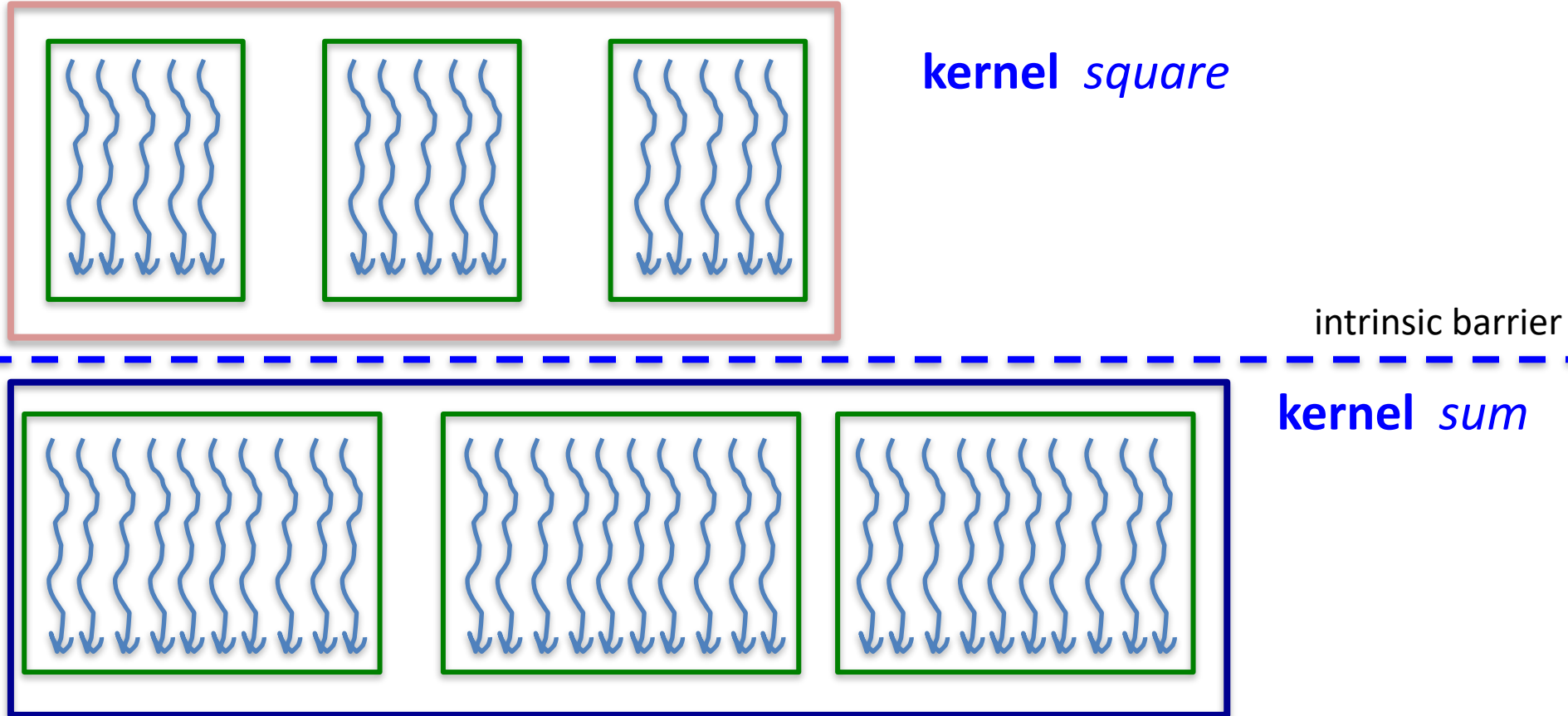
    cudaDeviceSynchronize();
    printf("End!\n");

    return 0;
}
```

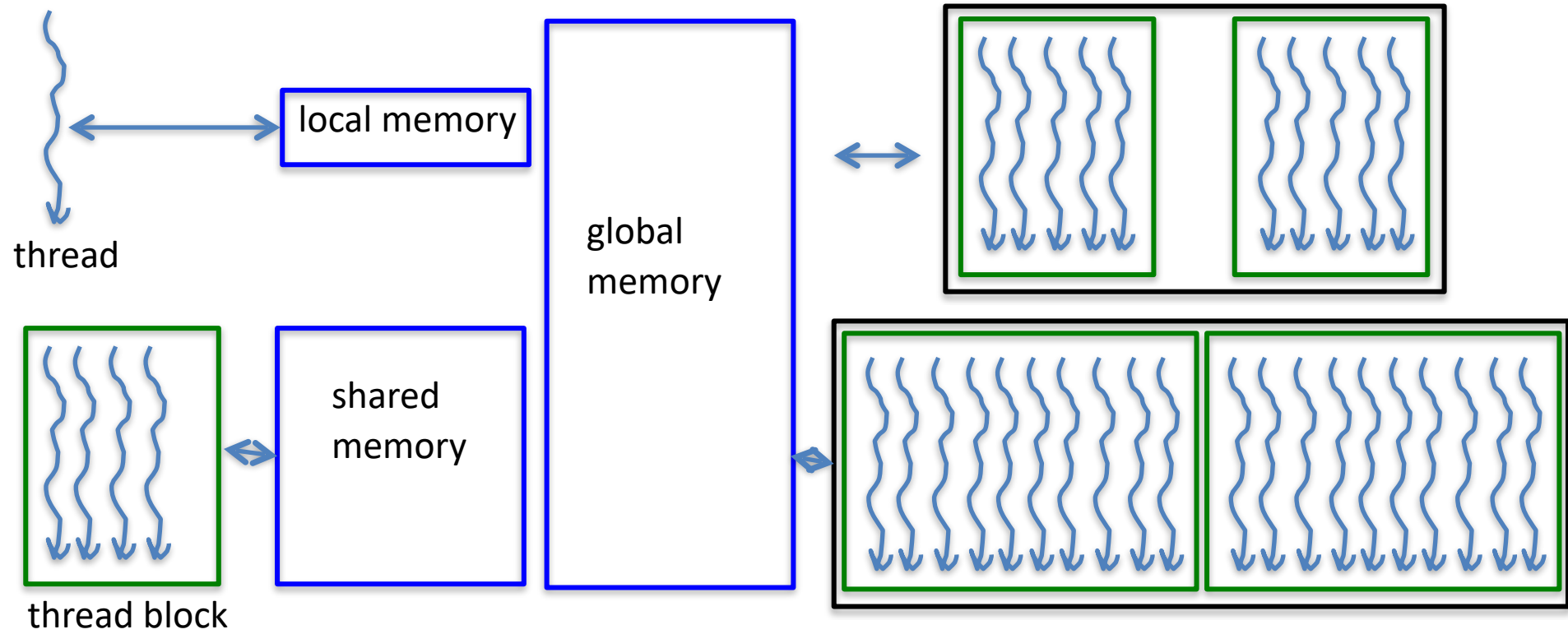

GPU Hardware

CUDA guarantees:

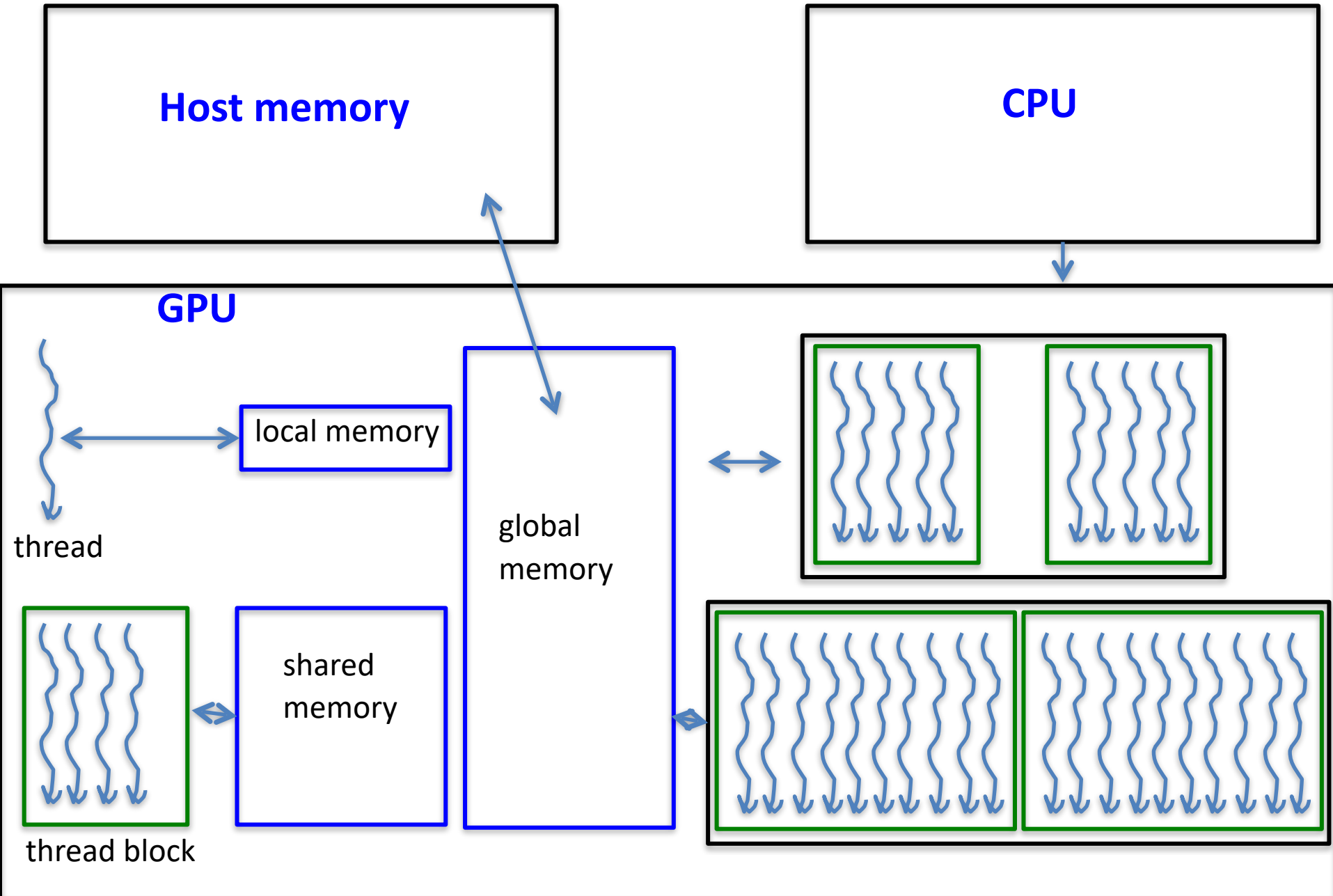
1. All threads in a block run on the same SM at the same time.
2. All blocks in a kernel finish before any blocks from the next kernel run.



Memory model



Memory model



GPU Hardware

Questions:

1. All threads from a block can access the same variable in that blocks shared memory
2. Threads from 2 different blocks can access the same variable in global memory
3. Threads from different blocks have their own copy of local variables in local memory
4. Threads from the same block have their own copy of local variables in local memory

Synchronization

Threads can access each other's results through global and shared memory

DANGER: What if one thread tries to read the result before the other one writes it?



...

```
int idx=threadIdx.x;
__shared__ int array[128];

array[idx]=idx;

if (idx<127){
    array[idx]=array[idx+1];
}
...
```

Synchronization

Threads can access each other's results through global and shared memory

DANGER: What if one thread tries to read the result before the other one writes it?



...

```
int idx=threadIdx.x;  
__shared__ int array[128];
```

```
array[idx]=idx;  
__syncthreads();
```

```
if (idx<127){  
    array[idx]=array[idx+1];  
}
```

...

Synchronization

Threads can access each other's results through global and shared memory

DANGER: What if one thread tries to read the result before the other one writes it?



...

```
int idx=threadIdx.x;
__shared__ int array[128];

array[idx]=idx;
__syncthreads();

if (idx<127){
    int temp = array[idx+1];
    __syncthreads();
    array[idx]=temp;
}

...
```

Synchronization

Threads can access each other's results through global and shared memory

DANGER: What if one thread tries to read the result before the other one writes it?



...

```
int idx=threadIdx.x;
__shared__ int array[128];

array[idx]=idx;
__syncthreads();

if (idx<127){
    int temp = array[idx+1];
    __syncthreads();
    array[idx]=temp;
}

...
```